Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3725843.3756054

RESEARCH-ARTICLE

OASIS: A Commercial High Performance Terminal AI Processor Supporting RISC-V Tensor Extension Instructions

PENG GAO, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

YANG LIU, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

HAONAN SUN, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

JIANG JIANG

JUN WANG

ZONGHUI HONG

View all

Open Access Support provided by:

Beijing University of Posts and Telecommunications

PDF Download
3725843.3756054.pdf
11 March 2026
Total Citations: 1
Total Downloads: 4040

Published: 17 October 2025

Citation in BibTeX format

MICRO 2025: 58th IEEE/ACM International Symposium on Microarchitecture
October 18 - 22, 2025
Seoul, Korea

Conference Sponsors:
SIGMICRO

https://doi.org/10.1145/3725843.3756054
ISBN: 9798400715730

OASIS: A Commercial High Performance Terminal AI Processor Supporting RISC-V Tensor Extension Instructions

Peng Gao
Beijing University of Posts and Telecommunications
Beijing, China
SOPHGO TECHNOLOGIES LTD.
Beijing, China
peng.gao@bupt.edu.cn

Yang Liu*
Beijing University of Posts and Telecommunications
Beijing, China
liu.yang@bupt.edu.cn

Haonan Sun
Beijing University of Posts and Telecommunications
Beijing, China
haonan.sun@bupt.edu.cn

Jiang Jiang
SOPHGO TECHNOLOGIES LTD.
Beijing, China
jiang.jiang@sophgo.com

Jun Wang
SOPHGO TECHNOLOGIES LTD.
Beijing, China
james.wang@sophgo.com

Zonghui Hong
SOPHGO TECHNOLOGIES LTD.
Beijing, China
zonghui.hong@sophgo.com

Jiali Qu
SOPHGO TECHNOLOGIES LTD.
Beijing, China
jiali.qu@sophgo.com

Abstract

The exponential growth of artificial intelligence (AI) has spurred a demand for efficient, scalable computing. While domain-specific accelerators (DSAs) outperform general-purpose GPUs, their closed architectures lead to ecosystem fragmentation. The open RISC-V ISA offers a path to a unified ecosystem, but faces fundamental architectural challenges: the inefficiency of standard extensions for domain-specific workloads and severe memory contention in heterogeneous SoCs. This paper investigates these challenges and proposes a set of evidence-backed design principles for high-performance RISC-V AI accelerators. To this end, we present OASIS, the first commercial terminal AI processor supporting RISC-V tensor extension instructions. Its core innovation is a scalable, multi-core RISC-V Tensor Processing Unit (RTPU) architected with three key principles: a hardware-accelerated data path to solve the `im2col` bottleneck; a decoupled memory subsystem to eliminate resource contention; and a novel, low-overhead synchronization mechanism that enables near-linear multi-core scaling. The success of this evidence-driven architecture is ultimately validated by an evaluation of the fabricated chip. To demonstrate its versatility, we benchmark its performance across a wide spectrum of representative AI models. Our results consistently show superior performance over leading commercial platforms across key terminal AI scenarios. This outcome is supported by deep-dive analyses including multi-core scaling and multi-model concurrency tests, alongside a Roofline model which confirms high hardware utilization. By submitting our tensor extension for standardization, this work establishes a new, evidence-backed blueprint for an open, scalable, and high-performance AI computing ecosystem.

CCS Concepts

• **Computer systems organization** → **Architectures**; *Special purpose systems*; *Reduced instruction set computing*; • **Computing methodologies** → *Machine learning*; • **Hardware** → *Hardware-software codesign.*

Keywords

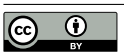
AI processor, RISC-V, tensor extension instruction, RTPU, microarchitecture, toolchain

ACM Reference Format:
Peng Gao, Yang Liu, Haonan Sun, Jiang Jiang, Jun Wang, Zonghui Hong, and Jiali Qu. 2025. OASIS: A Commercial High Performance Terminal AI Processor Supporting RISC-V Tensor Extension Instructions. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 20 pages. https://doi.org/10.1145/3725843.3756054

1 Introduction

During recent decades, general purpose GPUs (GPGPUs) have played a key role in meeting the growing computational demands of artificial intelligence (AI) applications[3, 22, 28, 35, 39, 45–48, 56]. These GPGPUs, such as NVIDIA's T4, A100, and H100, fully utilize their single instruction multiple thread (SIMT) architecture and innovative microarchitectures such as Tensor cores, providing the necessary performance and flexibility to handle deep learning training and inference tasks. However, GPGPUs often face challenges in optimizing cost efficiency and energy efficiency, particularly as the scale and diversity of AI workloads continue to expand [4, 11, 14, 54, 55, 60], emphasizing the need for specialized computing solutions.

*Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License.
MICRO '25, Seoul, Republic of Korea
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1573-0/25/10
https://doi.org/10.1145/3725843.3756054

In recent years, the pursuit of higher cost-efficiency and energy efficiency has spurred extensive research and development of domain-specific AI accelerators [7, 33, 36, 37, 41, 43, 51–54, 60, 67]. These accelerators, such as Google's Tensor Processing Unit (TPU) [29–31] and Groq's Language Processing Unit (LPU) [1], with AI as their sole target application, adopt simple instruction sets and innovative architectures. These specialized accelerators have shown improvements in cost efficiency and energy efficiency compared to GPGPUs [2, 16, 17, 21], addressing the growing demands of AI workloads more effectively.

Despite their promising potential, traditional AI-specific accelerators still suffer from two key limitations: **Fragmented ecosystems due to closed architectures**. Most traditional accelerators are built on proprietary and incompatible instruction sets and architectural designs, leading to a high engineering overhead to adapt them to mainstream AI frameworks [5, 8]. **Limited openness and scalability.** The closed design and lack of standardization hinder interoperability, making it difficult to establish an interconnected, scalable, and cohesive AI hardware ecosystem, ultimately hindering the widespread use of traditional accelerators [23].

Finding a unified and scalable instruction set for various artificial intelligence workloads has been recognized as a critical issue. Currently, more and more academic and industry teams are preparing the answer to this question with RISC-V [9, 18]. RISC-V has the characteristics of freedom, openness, and modularity. Freedom refers to anyone being able to use the RISC-V instruction set for free [40]. Openness refers to the ability to add customized instruction sets to the RISC-V instruction set according to demand [42, 66]. Modularity refers to the option for processors designed for specific applications to support only a portion of RISC-V instructions.

We believe that the RISC-V instruction architecture, with its numerous advantages, can serve as the foundational instruction set and architectural solution to meet the computing demands of rapidly evolving artificial intelligence workloads. Using RISC-V customized extension instructions, various AI computing workloads can be efficiently processed [15]. The rapid development of AI has resulted in diverse application scenarios, which can be geographically classified into cloud and terminal environments, and by data type into visual, linguistic, speech, and multimodal applications [25, 27, 38, 68]. Using the modular nature of RISC-V, the optimal instruction set can be selected for a given application scenario to achieve maximum efficiency [19, 44].

Unfortunately, the adoption of RISC-V for AI-specific accelerators faces the following challenges at present: **Low efficiency for multimodal AI workloads.** RISC-V's Instruction Set Architecture (ISA) including vector extension and potential matrix extension has low efficiency for multimodal AI workloads [12], such as convolution. **Resource contention for accessing memory resources.** Integrating general-purpose computing cores with extended instruction computing cores poses challenges in terms of memory access bandwidth resource allocation and memory access latency [13, 49, 59].

To overcome these challenges, we perform a rigorous pre-design analysis of existing RISC-V AI paradigms. Our analysis reveals critical trade-offs in each approach. Both vector-centric approaches (like the SiFive X280 [58]) and matrix co-processor approaches (like Tesla's DOJO [10]), while flexible, are fundamentally inefficient for multimodal AI model. Our modeling shows that because they leave the expensive data rearrangement (`im2col`) task to software, this overhead consumes 35% of the total execution cycles as summarized in Table 1. In parallel, the radical dataflow and tensor streaming processor (TSP) architecture pioneered by Groq [1], achieves exceptional, deterministic performance on certain workloads by relying on massive on-chip SRAM over 230MB to eliminate DRAM bottlenecks. However, this architectural choice trades away the dynamic flexibility and cost-effectiveness required for diverse, concurrent terminal AI tasks and strictly limits the maximum model size supported by a single chip. This analysis makes our design trajectory clear: a truly versatile terminal accelerator must solve the data rearrangement bottleneck in hardware without sacrificing the programmability required for dynamic workloads. To validate a potential solution, our own modeling predicted that a specialized architecture with a hardware-first data path could drastically alter the performance landscape. Our model shows that such a design can reduce the data management overhead to less than 5%, allowing the hardware to spend over 95% of its time on useful computation.

Based on this compelling evidence, we design OASIS, the first commercial high-performance terminal AI processor that supports RISC-V tensor extension instructions. The RISC-V Tensor Processing Unit (RTPU) in OASIS embodies this balanced design philosophy through an innovative and scalable multi-core architecture. This architecture is built on three key principles: a hardware-accelerated data path that performs `im2col` implicitly to solve the primary data movement bottleneck; a decoupled memory subsystem featuring a separate Load & Store Unit (LSU) to architecturally eliminate resource contention; and a scalable, flexible multi-core design enabled by a novel, low-latency synchronization mechanism to handle modern, concurrent AI tasks. This entire hardware system is supported by a complete toolchain that automates graph optimization, quantization, and compilation from standard frameworks via ONNX to ensure models run efficiently on the RTPU.

The success of this evidence-driven architecture is ultimately validated by an evaluation of the fabricated OASIS chip. To demonstrate its versatility, we benchmark its performance across a wide spectrum of representative AI models, spanning from efficient mobile networks to large-scale generative models. Our results consistently show state-of-the-art performance across three key scenarios: a multi-batch CNN, a single-batch Transformer, and a real-time object detector. This performance is supported by deep-dive analyses including multi-core scaling and multi-model concurrency tests, alongside a Roofline model which confirms high hardware utilization. Together, these results provide strong empirical evidence for our architectural principles and establish OASIS's potential as a unified, high-performance platform for next-generation AI.

The contributions of this paper are summarized as follows:

•We identify and provide a quantitative analysis of the fundamental architectural challenges limiting current RISC-V AI accelerators, specifically the computational inefficiency of standard extensions for workloads requiring complex data rearrangement and the critical memory contention bottleneck in heterogeneous SoCs.

•We design OASIS, the first commercial AI-specific accelerator based on RISC-V, embodying a novel, scalable multi-core architecture that incorporates a tensor extension instruction set and solves

Table 1: Cycle Cost Analysis for a Single ResNet-50 Convolutional Layer.

Architectural Paradigm	Primary Operation	Estimated Cycle Cost Breakdown
1. Vector-Centric (e.g., SiFive X280)	Software `im2col` + Vector MACs	**20%** Data Rearrangement, **80%** Compute
2. Matrix Co-processor (e.g., Tesla DOJO)	Software `im2col` + Hardware GEMM	**35%** Data Rearrangement, **65%** Compute
3. OASIS (Ours)	Hardware Data Gathering + Fused MAC	**<5%** Control Overhead, **95%** Compute

key bottlenecks with a hardware-accelerated data path and a decoupled memory subsystem, enabled by a specialized, low-overhead synchronization mechanism with demonstrably lower latency than traditional methods.

•We present an evaluation of the fabricated OASIS processor, providing strong empirical evidence from advanced techniques like Roofline analysis, multi-core scaling and multi-model concurrency tests that validates our architectural principles and demonstrates state-of-the-art performance and versatility.

• We submit our proposed solution to the RISC-V International Technical Steering Committee for standardization, aiming to establish it as a new, evidence-backed blueprint for the design of open AI-specific processors.

2 SoC

OASIS is a high-performance system-on-chip (SoC) designed to enable efficient execution of multi-modal AI workloads alongside high-performance general-purpose computing. This section details its key specifications, starting from the physical implementation and moving to its core computational subsystems as shown in Fig. 1.

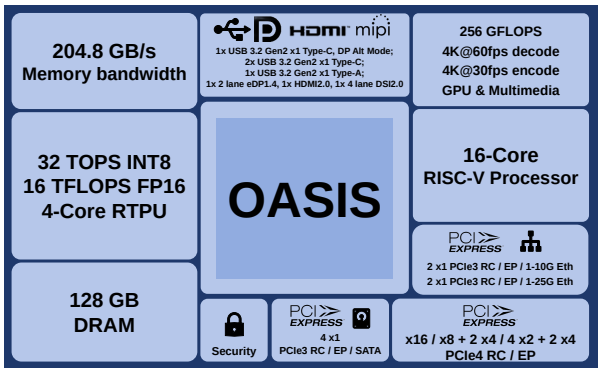


Figure 1: Functional Block of OASIS.

2.1 Physical Implementation and System-Level Features

Packaging and Thermal Management: The chip is fabricated on the 12nm process technology and is housed in a Flip-Chip Ball Grid Array (FCBGA) package for robust integration. OASIS operates within a typical power envelope of 35W and is supported by a thermal solution with a Thermal Design Power (TDP) of 50W, consisting of an integrated heat spreader (IHS) and an active cooling assembly to ensure sustained performance without thermal throttling.

Memory and I/O Subsystems: The foundation of the SoC's performance is its high-bandwidth memory system, supporting 256-bit LPDDR5 with a maximum bandwidth of 204.8 GB/s and a capacity of up to 128 GB. This large memory capacity and bandwidth are essential for supporting the large-scale AI models targeted by OASIS. To ensure seamless connectivity, the SoC also integrates a set of high-speed interfaces, including PCIe, SATA, Ethernet, and USB.

2.2 Core Computational Clusters

General-Purpose CPU Cluster: OASIS features a 16-core RISC-V high-performance processor complex running at 2 GHz. Supporting the latest RVA22, hypervisor, and vector extensions, it delivers competitive general-purpose performance (SPECINT2006 rate of 12/GHz) and serves as the main controller for the system.

AI Processing Subsystem (RTPU): The standout feature of OASIS is its RISC-V Tensor extension-based 4-core RTPU subsystem, which is the primary focus of this work. Operating at 1 GHz, the RTPU provides a peak performance of 32 TOPS INT8 and 16 TFLOPS FP16, and is specifically designed for accelerating diverse multi-modal AI inference tasks.

2.3 Multimedia Acceleration and Physical Layout

GPU and Multimedia Engines: To complement its AI capabilities, the processor includes a desktop-grade GPU (256 GFLOPS) and dedicated hardware for 4K video encoding and decoding. These capabilities, combined with interfaces for ISP and CSI, make OASIS an ideal, all-in-one platform for complex multimedia applications.

Layout Overview and Area Breakdown: The OASIS die has a total area of 451 mm^2. The physical layout prioritizes the AI acceleration capabilities, with the core RTPU subsystem occupying 97.2 mm^2 (21.5% of the total die). Within the RTPU, the four powerful TPU cores consume 75.9 mm^2, while the four controlling SVPU cores and their shared L3 cache take up 13.1 mm^2, as detailed in Fig. 2.

3 RTPU Subsystem

To address the growing computational demands of multimodal AI applications, the RISC-V Tensor Processing Unit (RTPU) subsystem integrates scalar, vector, and tensor computation capabilities into a unified architecture. By coupling SVPU with TPU, the RTPU balances computational flexibility and performance efficiency. The SVPU extends standard RISC-V CPUs with advanced vector processing capabilities, while the TPU acts as a domain-specific accelerator optimized for high-dimensional tensor operations. This design enables the RTPU to efficiently process diverse workloads, ranging

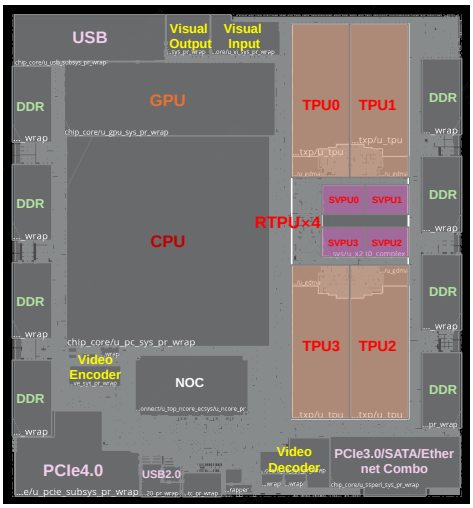


Figure 2: Die photo of OASIS.

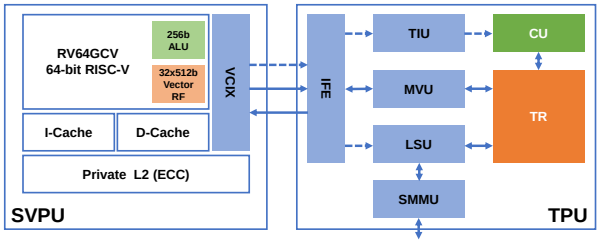


Figure 3: Architecture of RTPU.

from traditional scalar&vector computations to large-scale tensor-based deep learning tasks.

3.1 Architecture of RTPU

The RTPU architecture is designed to seamlessly integrate scalar, vector, and tensor processing within a single cohesive framework, enabling high-performance execution of AI workloads. Each RTPU core consists of an SVPU and TPU, connected through VCIX, as shown in Fig. 3. The SVPU manages the fetching, decoding and execution of scalar and vector instructions, while also dispatching tensor instructions to the TPU. The TPU accelerates tensor-based computations using its high computing power and high-capacity tensor registers. This tightly coupled design allows the RTPU to efficiently support both low- and high-dimensional data processing, addressing the diverse computational demands of AI applications. Multiple RTPU cores can be organized into a core complex, with shared L3 cache to ensure memory consistency for SVPU operations, while TPU memory consistency is managed at the software level.

3.1.1 Architecture of SVPU. Each RTPU core contains a Scalar and Vector Processing Unit (SVPU), which acts as the main control processor. The SVPU is a RISC-V CPU core supporting the RV64GCV instruction set, featuring a standard multi-level cache hierarchy with 32 KB L1 caches, a 256 KB private L2 cache, and a 2 MB L3 cache shared among four SVPUs. It is designed to handle all scalar and vector operations, and most importantly, to dispatch tensor instructions to its companion TPU.

The crucial component enabling this heterogeneous operation is the Vector Core Interface Extension (VCIX), as illustrated in Fig. 4. The VCIX is a high-throughput, decoupled interface that facilitates efficient data exchange between the SVPU's Vector Register File (VRF) and external modules like the TPU. Its key features are:

Decoupled Channels. The interface is structured with independent request and response channels governed by standard Valid/Ready flow control. This decoupling allows the SVPU and TPU to operate asynchronously, maximizing parallelism.

Request Channel. This channel transfers high-level control information and operands from the SVPU to the TPU. It conveys both standard RISC-V vector parameters (like SEW, LMUL, and vl) and the full operand data for our custom tensor instructions.

Response Channel. This channel handles the transfer of data from the TPU back to the SVPU's VRF. It is designed to support high-bandwidth, multi-cycle data transfers to accommodate large tensor results.

This tightly-coupled yet decoupled interface ensures efficient communication between the SVPU and TPU, enabling the RTPU subsystem to seamlessly process a wide range of computational tasks, from standard vector operations to complex, tensor-based AI workloads.

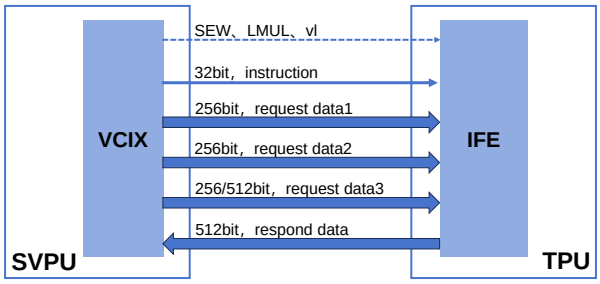


Figure 4: VCIX Interface.

3.1.2 Architecture of TPU. From an architectural perspective, the TPU is a highly specialized processor composed of several key subsystems. Its architecture is organized around two primary functional planes: a control and dispatch plane that manages the instruction flow, and a compute and host interface plane that handles data processing and exchange.

Control and Dispatch Plane. The Instruction Front End (IFE) serves as the gateway to the TPU. It receives and decodes VCIX commands from the SVPU, checks for data dependencies, and dispatches them to the appropriate execution units. The IFE's register-centric design, which manages a rich set of configuration and metadata registers (CSRs, TVRs/GVRs), enables precise, in-order control over all tensor operations. The Tensor Instruction Unit (TIU) works in tandem with the IFE, generating the fine-grained control signals and read/write strobes for the compute and memory units, orchestrating the parallel execution in an SIMD (Single Instruction, Multiple Data) manner.

Compute and Host Interface Plane. The computational heart of the TPU is its array of 32 Compute Units (CUs). Each CU is a heterogeneous engine containing both a Cube Unit for high-throughput INT8/BF16/FP16 matrix multiplication (performing 4×32×32×1 INT8 MACs per cycle) and a Vector Unit for flexible subword-parallel operations. This allows 4D tensor operations to be decomposed into the most efficient underlying 1D or 2D compute primitives. Finally, the Move Unit (MVU) facilitates data exchange between the TPU's internal TRs and the host SVPU's Vector Register File (VRF), providing the critical data path for CPU-accelerator collaboration.

3.1.3 A Decoupled Memory Subsystem for Mitigating Contention. A key architectural challenge in heterogeneous SoCs is that when a high-throughput AI accelerator is integrated with general-purpose CPU cores, they inevitably compete for shared memory resources, creating a critical performance bottleneck. A pivotal design decision is whether the accelerator should share the host CPU's LSU and memory path, or possess its own.

To quantify this risk, we explore two architectural models using our cycle-accurate simulator: a shared-LSU architecture and our proposed decoupled architecture. We evaluate them under a realistic mixed-workload scenario where a memory-intensive CPU task (a "stream-triad" benchmark) runs concurrently with a ResNet-50 inference task on the accelerator. As summarized in Table 2, the data clearly demonstrates that a shared-LSU architecture suffers from crippling performance degradation of approximately 80% when the memory system is under contention.

Table 2: AI Task Performance Degradation under Mixed Workload.

Memory Architecture	AI Throughput Degradation	Primary Bottleneck
Shared-LSU Architecture	~80%	LSU Port Contention
OASIS Decoupled Architecture	**<5%**	**DRAM Controller Arbitration**

Based on this compelling evidence, we design the OASIS TPU with a fully decoupled memory subsystem to architecturally eliminate this bottleneck. This subsystem is comprised of three key, synergistic components. At its core is a 4MB on-chip Tensor Register (TR) file, which acts as a software-managed scratchpad with a heterogeneous design to efficiently store both intermediate feature maps and persistent weights. All data transfers between this on-chip storage and DRAM are managed by a dedicated LSU, which crucially incorporates advanced hardware functionalities like on-the-fly transpose and broadcasting to offload the compute units. Finally, a dedicated System Memory Management Unit (SMMU) handles all virtual-to-physical address translation for the TPU, fully isolating its memory address space from the host CPU.

This architectural choice isolates the TPU's high-bandwidth data streaming from the SVPU's traffic, with the only point of contention being the final DRAM controller. From our post-layout analysis, the area cost for the dedicated LSU and SMMU is a modest 1.7 mm^2, and it is not on the critical timing path. By investing this small area, we mitigate a potential 75% throughput degradation, ensuring sustained and predictable AI performance in complex terminal applications.

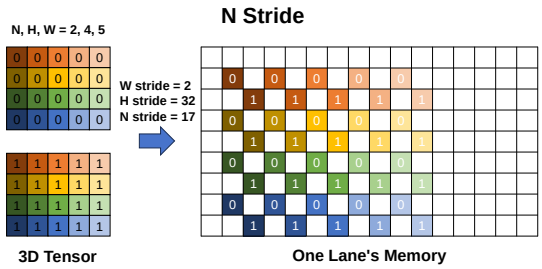


Figure 5: 3D Tensor's Layout in One STR.

3.2 Programming Model

The programming model of the RTPU Subsystem defines the framework through which computational tasks are efficiently executed on the hardware. It encompasses the storage model, programming methodologies, and synchronization mechanisms necessary for optimal performance.

3.2.1 Storage Model. The TPU's storage model is architected to efficiently handle the 4D tensors fundamental to AI workloads. It defines a clear hierarchy and addressing scheme for data residing both in off-chip DRAM (as Global Tensors) and in the on-chip Tensor Register (TR) file. This is crucial for enabling complex data access patterns with high performance.

Hierarchical Tensor Representation. Our model defines two levels of tensor representation. At the highest level, Global Tensors (GTs) represent the complete 4D tensors (N, C, H, W) residing in main DRAM. Internally, the 4MB of on-chip TRs are organized into smaller banks known as Slice Tensor Registers (STRs). As conceptually illustrated in Fig. 5, each STR can hold a 2D or 3D slice of a larger tensor, with its layout governed by a flexible, stride-based addressing scheme.

Key Architectural Benefit: Zero-Overhead Slicing and Grouping. The primary advantage of this stride-based model is that it enables zero-overhead data slicing. The compiler can define and access complex tensor slices-such as those needed for grouped convolutions or depth-wise operations-simply by programming the appropriate stride and offset parameters into the hardware's metadata registers. This completely avoids the need for expensive, explicit data copy or rearrangement operations in software, which would otherwise be a major performance bottleneck.

Host Access via LMEM Abstraction. To facilitate physical implementation and communication with the host SVPU, collections of STRs are logically grouped into 32 banks of Local Memory (LMEMs), as shown in Fig. 6. Each LMEM bank presents a simple, contiguous 128KB address space to the SVPU. This abstraction allows the host processor to efficiently exchange data with the TPU's on-chip memory using standard MOVE instructions, without needing to understand the complex internal STR layout. This modular memory structure enhances both the programmability and the versatility of the system.

3.2.2 Programming Methods. The programming model for the RTPU subsystem is designed to be both powerful and flexible, supporting efficient execution on a single core as well as scalable parallelism across multiple cores.

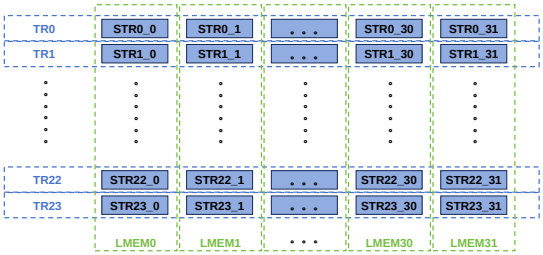


Figure 6: The relationship among TRs, STRs and LMEMs.

Single-Core Programming: A Heterogeneous Model. Programming a single RTPU core follows a heterogeneous model where tasks are dispatched to the functional unit best suited for them. Standard computations, control flow, and small-scale parallel tasks are handled by the SVPU, where developers can leverage the full power of the RISC-V scalar and Vector (RVV) instruction sets. This can be done either directly through assembly and intrinsic functions, or by relying on the compiler's auto-vectorization capabilities. In contrast, computationally-intensive AI workloads, such as matrix multiplication, convolution, and quantization, are offloaded to the TPU via our specialized Tensor Extension Instructions. These instructions operate on 4D tensors and are issued from the SVPU, which acts as the control center for the entire tensor processing task. This clear division of labor ensures that each part of an application runs on the most efficient hardware.

Multi-Core Programming: Scalable Parallelism. To leverage the full power of the 4-core RTPU complex, we support standard parallel programming models familiar to CPU developers. Developers can employ a Fork-Join model for task-level parallelism or a Single Program Multiple Data (SPMD) model for data-level parallelism. This allows a single AI model to be efficiently partitioned and executed across all four RTPU cores, or for multiple independent models to run concurrently. The choice of model depends on the specific application's requirements, and this flexibility is key to fully exploiting the RTPU's multi-core processing capabilities. The underlying synchronization mechanisms are detailed in the following section.

3.2.3 Synchronization Methods. The RTPU's architecture, where tensor instructions are interspersed with standard RISC-V instructions, requires robust synchronization at multiple levels: within a single core to ensure data consistency, and between multiple cores to enable efficient parallel execution.

Intra-RTPU Synchronization. Within a single RTPU core, the compiler is responsible for ensuring correct data dependencies between the SVPU and TPU. It achieves this by inserting a `sync.i` instruction where necessary. This instruction acts as a barrier, stalling the SVPU until all previously dispatched tensor operations (e.g., Load/Store, Compute) have been completed by the TPU. This explicit, compiler-managed approach guarantees deterministic execution for mixed-instruction workloads.

Inter-RTPU Synchronization: A Novel Low-Overhead Mechanism. Synchronizing multiple RTPU cores for collaborative tasks presents an architectural challenge. Traditional multi-core CPU synchronization methods, which rely on cache-coherent atomic instructions (`AMO`) and memory barriers (`FENCE`), are ill-suited for this purpose. Such methods involve bus locking, multi-level cache consistency maintenance bus operations, multi-level cache flushing and other operations, resulting in high non-deterministic latencies, which would severely degrade the performance of a tightly-coupled accelerator array.

Table 3: Average Latency of a Single Synchronization Event.

Synchronization Method	Latency (Cycles)	Primary Bottleneck
Cache-Coherent Atomics (AMO)	~100-200	Coherence Protocol, Bus Contention
OASIS Message Queue	**~25**	**AXI Bus Round-Trip Time**

To address this, we design and implement a lightweight, message-based counting semaphore mechanism. This system bypasses the cache hierarchy entirely, using a dedicated hardware message queue. The core logic is illustrated in the pseudocode in Fig. 7. Our pre-design analysis, which compared this approach to traditional methods, quantitatively proves its superiority, as summarized in Table 3.

The data clearly shows that our dedicated hardware messaging system provides the lowest and most predictable latency. This system uses a 128-entry message queue accessed via non-cacheable `msgsend` and `msgwait` instructions. The 128-entry depth is selected after analyzing the dependency graphs of various parallelized AI models to prevent backpressure without area overhead. Our post-layout analysis confirms this entire module occupies a minimal area of just 0.08 mm^2 and meets our 1 GHz timing target.

This evidence-driven design choice is a key contribution of our work. By investing a small amount of area in this dedicated synchronization mechanism, we achieve a 4-8× reduction in synchronization latency compared to traditional methods, which is critical for enabling the near-linear performance scaling demonstrated in our evaluation (Section 5.3.1).

```
1  /*the processing function for the GIVE message after
   it is received by the message queue*/
2  def recv_give(signal_id, wait_cnt){
3    /*When the first GIVE message is received,
     initialize the remain_wait_cnt variable*/
4    if (signal_que[signal_id].remain_wait_cnt==0)
5      signal_que[signal_id].remain_wait_cnt=wait_cnt;
6      /*for every GIVE message received, increment
       sent_cnt by one*/
7      signal_que[signal_id].sent_cnt ++;
8   }
9  /*the processing function for the WAIT message after
     it is received by the message queue*/
10 def recv_wait(signal_id, send_cnt){
11   while (signal_que[signal_id].sent_cnt != send_cnt)
12     continue;
13   /*send a TAKE message once the current queue's
     send_cnt matches the send_cnt specified in the WAIT
     message*/
14   send_take(rtpu_id, success);
15   /*if remain_wait_cnt is zero, clear the information
     associated with that ID*/
16   if (--signal_que[signal_id].remain_wait_cnt == 0)
17     signal_que[signal_id] = {0};
18 }
```

Figure 7: Pseudo Code for Synchronization Method for Multiple RTPUs.

3.3 Coordination of Vector and Tensor Instructions

A critical aspect of the RTPU's design is ensuring the correct and hazard-free execution of mixed instruction streams containing both standard RVV instructions and our custom Tensor instructions, as shown in Appendix. The coordination is managed through a combination of architectural decoupling, an explicit synchronization instruction.

3.3.1 Architectural Decoupling via VCIX. The SVPU (which executes scalar and RVV instructions) and the TPU (which executes tensor instructions) are architecturally distinct units connected via the VCIX interface. The SVPU's role is to fetch and decode all instructions. It executes scalar and RVV instructions in its own pipeline, while dispatching tensor instructions as commands to the TPU via the VCIX's request queue. The TPU executes these commands asynchronously. This decoupling is fundamental to preventing structural hazards on the main execution pipelines.

3.3.2 Register File Arbitration and Hazard Handling. The primary shared resource between the SVPU and TPU is the Vector Register File (VRF), which is physically part of the SVPU. Tensor `MOVE` instructions initiate data transfers between the VRF and the TPU's internal LMEM. To prevent hazards, we employ a two-level approach. First, for structural hazards at the register file port level, the VCIX interface itself has hardware interlocks. When a `t.move` instruction is dispatched, the VCIX logic arbitrates for access to the required VRF ports, and the SVPU's pipeline will stall if it attempts to access a port already in use by the VCIX. Second, for data dependency hazards (like Read-After-Write (RAW)), the onus is on the software (compiler) to ensure correctness via an explicit synchronization model, inserting a `sync.i` instruction. This instruction acts as a full execution barrier, again stalling the SVPU pipeline until the TPU confirms that all previously dispatched tensor instructions are complete. This stall behavior is the core of our pipeline hazard management: the instruction decoder ceases to issue new operations, effectively freezing the pipeline until the hazard is resolved, after which it resumes cleanly without needing a complex flush-and-refetch mechanism. This entire process guarantees data consistency with minimal hardware complexity.

3.3.3 Example of Hazard Management. Consider the following pseudo-assembly sequence as shown in Fig. 8, which illustrates how a potential RAW hazard is resolved:

```
1 ls.ld td0, gs0 #TPU begins loading data from DRAM into
                  its LMEM
2 ls.ld td1, gs1 #TPU begins loading data from DRAM into
                  its LMEM
3 ls.ld td2, gs2 #TPU begins loading data from DRAM into
                  its LMEM
4 conv td3, td0, td1, td2, _rq #TPU begins a convolution
                                on the data
5 mov.v.t v8, (r1) #TPU moves the result from LMEM to
                    VRF register
6 sync.i #<-- CRITICAL BARRIER
7 vadd.vv v9, v8, v1 #SVPU uses the result in v8 for a
                      vector addition
```

Figure 8: Pseudo-Assembly Sequence.

In this example, the RVV instruction `vadd.vv` depends on the result produced by the tensor instructions. Without the `sync.i` barrier at line 6, the SVPU could execute line 7 prematurely, before the `move.v.t` instruction has finished writing the result to register `v8`. The `sync.i` instruction forces the SVPU to wait until the TPU signals completion, thus guaranteeing that `v8` contains the correct data and resolving the RAW hazard. Our compiler is responsible for automatically inserting these synchronization instructions to ensure program correctness. This explicit model is chosen over a more complex hardware-based out-of-order scoreboard between the two disparate units, as it provides deterministic behavior with lower hardware complexity.

3.4 Microarchitecture of TPU

In this section, we will introduce key aspects of the TPU's microarchitecture through the process of convolution calculation, as shown in Fig. 9. Compared to matrix multiplication, convolution is a more complex operation, requiring additional microarchitectural consideration to achieve high performance. These optimizations are critical to ensuring efficient execution, especially given the computational and data movement demands inherent in convolutional operations.

3.4.1 Cube. In TPU, the convolution calculation process is divided into two steps: IMAGE to COLUMN and matrix multiplication. We first focus on the innermost loop, as illustrated by the code from line 19 to line 23 in the pseudocode, which describes the matrix multiplication performed during convolution computation at each clock cycle.

The matrix multiplication is carried out in 32 Cube Units in parallel as shown in Fig. 10. For INT8 data precision, each Cube Unit calculates the multiplication of a 4×32 left matrix (orange) and a 32×1 right matrix (green), and accumulates the result into a 4×1 matrix (blue). The right matrices in each Cube Unit are distinct, represented by different shades of green. The Cube Unit design uses four 32-to-1 adder trees, and the results of 32 multipliers in a column are accumulated in the adder tree, with purple blocks representing multipliers.

Another critical factor affecting performance and power consumption is minimizing data read and write operations, thus increasing data reuse. The TPU implements several design optimizations: during convolution calculation, the 4×32 left matrix is broadcasted to 32 Cube Units for parallel calculation, reusing the data 32 times. Furthermore, due to the ohw group (output height and width group) in the calculation process and the corresponding logic design, each 32×1 right matrix is reused 16 times once it is transmitted to the Cube Unit. The calculation results are temporarily stored in the Cube Unit's internal registers and accumulated multiple times before being written back to the TR.

3.4.2 LMEM. LMEM as shown in Fig. 11, is composed of 16 banks of SRAM, providing two read interfaces and two write interfaces. When there are no bank conflicts, different banks of the LMEM can be accessed simultaneously.

The STRs with the same suffix and even-odd-adjacent prefix of STR0_* to STR15_* share a bank of the LMEM with the same number of the suffix. For instance, STR0_1 and STR1_1 share bank

```
1 /*During each calculation, produce a total of 128
  results by computing 1 ON, 32 OC, and 4 OHW
  coordinates*/
2 for on in range(0, ON)
3   for oc in range(0, OC, 32)
4     /*under the same ON and OC dimensions, the results
      from 64 adjacent OHW coordinates are treated as
      one group*/
5     for ohw_grp in range(0, OH * OW, 64)
6       /*implement the multiply-accumulate operations
        related to the IC dimension using a 32-input
        adder tree*/
7       for ic in range(0, IC, 32)
8         for kh in range(0, KH)
9           for kw in range(0, KW)
10            /*traverse ohw_grp and compute 4 results
              at a time*/
11            for ohw in range(ohw_grp, ohw_grp + 64, 4)
12              oh[3:0] = F_oh(ohw + [3:0], OH, OW);
                //compute OH coordinate
13              ow[3:0] = F_ow(ohw + [3:0], OH, OW);
                //compute OW coordinate
14              /*compute IH coordinate*/
15              ih[3:0] = F_ih(oh[3:0], kh, IH, T_PADDING,
                B_PADDING, IH_INSERT, H_STRIDE, KH,
                KH_DILATION);
16              /*compute IW coordinate*/
17              iw[3:0] = F_iw(ow[3:0], kw, IW, L_PADDING,
                R_PADDING, IW_INSERT, W_STRIDE, KW,
                KW_DILATION);
18                /*32 Cube Units, each calculating 4
                  results*/
19                for i in range(0, 32)
20                  O[on, oc + i, oh[0],ow[0]] = I[on, ic:ic
                    + 32, ih[0], iw[0]] * K[ic:ic + 32,
                    oc + i, kh, kw] + O[on, oc + i,
                    oh[0], ow[0]];
21                  O[on, oc + i, oh[1],ow[1]] = I[on, ic:ic
                    + 32, ih[1], iw[1]] * K[ic:ic + 32,
                    oc + i, kh, kw] + O[on, oc + i,
                    oh[1], ow[1]];
22                  O[on, oc + i, oh[2],ow[2]] = I[on, ic:ic
                    + 32, ih[2], iw[2]] * K[ic:ic + 32,
                    oc + i, kh, kw] + O[on, oc + i,
                    oh[2], ow[2]];
23                  O[on, oc + i, oh[3],ow[3]] = I[on, ic:ic
                    + 32, ih[3], iw[3]] * K[ic:ic + 32,
                    oc + i, kh, kw] + O[on, oc + i,
                    oh[3], ow[3]];
```

Figure 9: Pseudo Code for the Convolution Calculation of TPU.

0 of LMEM1. Each STR of STR16_* to STR23_* independently access their corresponding bank in LMEM, where STR16_1 accesses bank 8 of LMEM1. This arrangement ensures efficient memory access across different STRs while minimizing bank conflicts.

3.4.3 Crossbar. In our architecture, the complex data gathering process required for convolutions, known in AI algorithms as "image to column" (`im2col`), is moved from software into dedicated hardware. This is essential for efficient convolution calculation. As shown in the pseudocode (lines 14-17), operations involving windows, strides, and padding necessitate reading non-contiguous data from the on-chip Slice Tensor Registers (STRs). The Crossbar is the microarchitectural unit designed to perform this gathering operation. During the convolution process, the high-performance dedicated computing logic in the TIU generates a read control signal for each LMEM in the LANE and a select control signal for

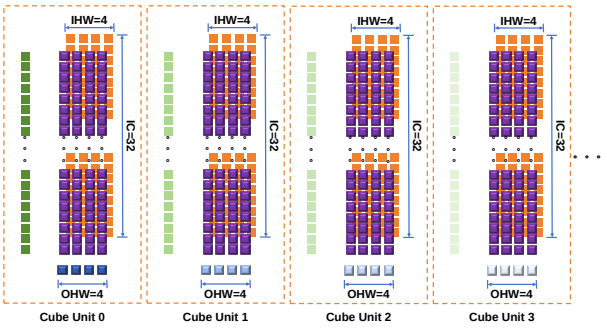


Figure 10: Matrix Multiplication in Cube Units in parallel.

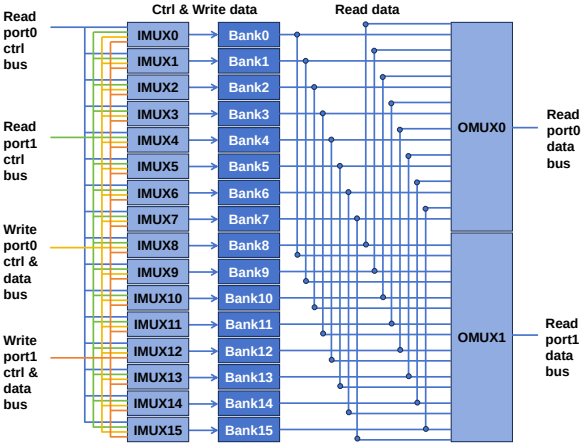


Figure 11: LMEM.

each clock cycle of the Crossbar. Data read from LMEM is routed through the Crossbar, which rearranges the elements as required before uploading the result to the Data-Gather Bus.

Fig. 12 illustrates the Crossbar's functionality. While its full throughput is utilized in operations like pooling or depth-wise convolution, for a standard INT8 convolution, it is configured to gather and output the 4 elements required from each of the 32 STRs per cycle.

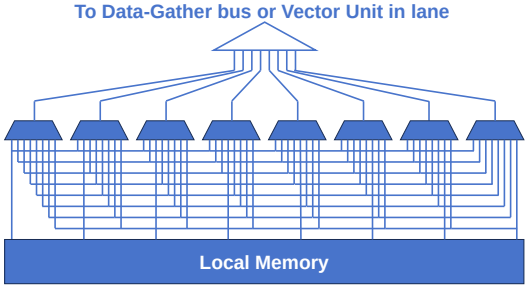


Figure 12: Crossbar.

The single-port read bandwidth for TR in TPU is 512 bytes (32 lanes × 16 bytes per lane). How to design the crossbar for such a large data throughput is an important issue. A key design decision is to implement this data gathering logic using a modular approach rather than a monolithic one. Our pre-design analysis shows that a single, monolithic 512×512×8bit crossbar would be prohibitively expensive. Initial synthesis estimates indicated it would require over 262K 8-bit multiplexers and, due to long wire delays across the ~10mm-wide of RTPU, would fail to meet our 1 GHz timing target.

Therefore, we adopt a modular, hierarchical design, which is a critical trade-off for achieving high performance in a physically realizable system. The logic is implemented as 32 smaller, independent 16×16 crossbars, one within each of the 32 lanes of the TPU microarchitecture. As illustrated in Fig. 12, this modular approach provides a 32× area reduction for the core logic (from ~262K MUXs to ~8K MUXs total) and ensures all internal paths are short and easily placed&routed to meet the 1 GHz clock frequency. This distributed crossbar design, working in conjunction with the global Data-Gather and Data-Broadcast Buses, provides the full `im2col` functionality at a fraction of the hardware cost of a monolithic implementation. The total post-layout area for all 32 crossbar modules is a modest 0.1 mm^2. To meet the 1 GHz clock target, this modular data path is deeply pipelined. The latency for a data element to traverse from an LMEM read port, through the Crossbar, and to the Data-Gather Bus is 4 clock cycles. It is critical to note that this is a pipeline latency, not a throughput limitation. The Crossbar is designed to sustain a throughput of one operation per cycle, meaning it can accept new data every cycle after the pipeline is filled. This initial latency is managed and hidden by the compiler's instruction scheduling, which ensures that data-gathering operations are initiated several cycles before the data is required by the compute units, thus having no impact on the sustained performance.

3.4.4 Lane. It is important to note that modules such as the Cube Unit, Vector Unit, Crossbar, and LMEM are each replicated 32 times in the TPU architecture. In the microarchitecture design, we group one Cube Unit, one Vector Unit, one LMEM, one data path (including the Crossbar), and the associated control logic into a unit called a *lane*, as shown in Fig. 13.

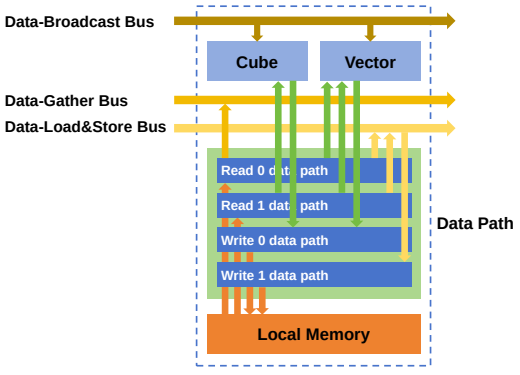


Figure 13: Lane.

Each LMEM has two read data ports and two write data ports, which results in four distinct data paths in each lane: the read 0 data path, the read 1 data path, the write 0 data path, and the write 1 data path. The Crossbar, as previously introduced, is a critical component in the read 0 data path. The data flow through these paths is indicated in Fig. 13, with the read 0 data path receiving input from LMEM and sending the output to either the Data-Gather Bus, the Data-Load & Store Bus, or the Vector Unit.

Lane's design adopts the idea of divide and conquer, which is an important factor in solving the problem of how TIU controls numerous computing and storage units, and another important factor is the corresponding global bus.

3.4.5 Global Ring Bus Network. The 32 distributed lanes of the TPU interconnect with the central Tensor Instruction Unit (TIU) via a Global Ring Bus Network. This network is crucial for coordinating data movement and control across the entire TPU microarchitecture. It consists of four main logical ring buses: the Data-Gather Bus for collecting results from the lanes, the Data-Broadcast Bus for distributing shared inputs, a Data-Load/Store Bus for data loading and storing from off-chip memory, and a Control-Broadcast Bus for instruction signals. An important feature of the Global Ring Bus Network is that the control commands and data reaching each lane are strictly scheduled by the TIU module, with no buffering or waiting anywhere in the entire ring network.

The operation of the two primary data buses is illustrated in Figs. 14 and 15. The Data-Gather Bus circulates through all lanes to collect partial inputs (e.g., 4 pieces of 8-bit data per lane during a convolution) and delivers them to the TIU. Subsequently, the TIU uses the Data-Broadcast Bus to send shared data back to all lanes, which is then fed directly into their compute units, bypassing LMEM storage to save power and bandwidth.

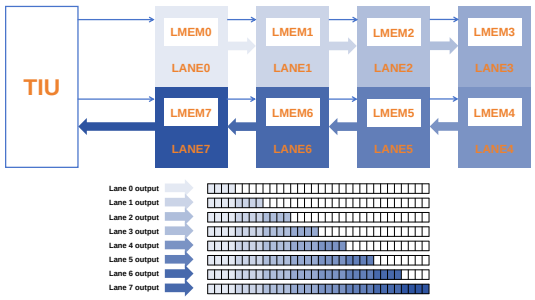


Figure 14: Data-Gather Ring Bus.

A key architectural challenge for this network is closing timing at our 1 GHz frequency target, given the long physical distances the buses must traverse across the die (up to 10 mm wide of RTPU).

Therefore, we make a critical, evidence-driven design choice to implement a dual-layer, heavily pipelined ring bus architecture, as conceptually shown in Fig. 16. This design breaks the long global paths into shorter, manageable segments connected by pipeline registers. While this approach adds pipeline latency and a modest area cost for the registers, it is an essential trade-off for achieving high-frequency operation in a large-die physical design. In addition, the dual-layer ring network can provide shorter latency access

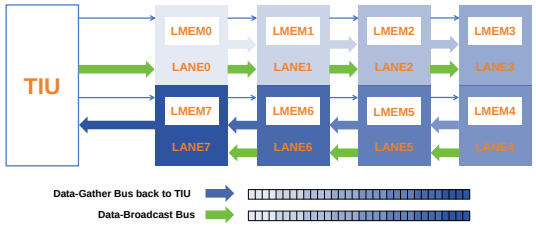


Figure 15: Data-Broadcast Ring Bus.

to resources within the target LANE than the single-layer ring network, thereby reducing the additional overhead of running AI models.

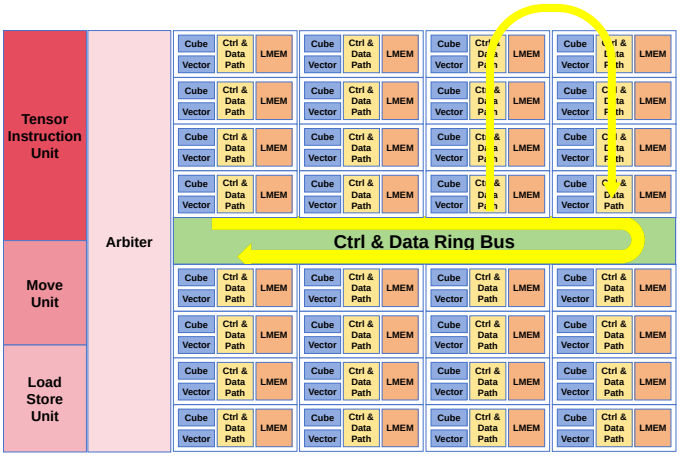


Figure 16: Dual Layer Ring Bus.

The associated pipeline registers of the entire global bus infrastructure, including all four main ring buses, occupies a total post-layout area of approximately 0.1 mm^2. This demonstrates that while the global communication network is a complex component, its hardware cost is manageable and necessary to enable the high-performance, distributed microarchitecture of the TPU. The end-to-end pipeline latency for a data packet to travel from the TIU to the furthest lane on the ring bus is 12 clock cycles. This latency is fully managed by our hardware's instruction scheduling, which issues data movement commands well ahead of their use, ensuring that it does not impact the sustained throughput of the processing lanes.

3.4.6 TIU Pipeline. The Tensor Instruction Unit (TIU) orchestrates the execution flow through a multi-stage pipeline, as illustrated in Fig. 17. The pipeline begins with a FETCH stage that retrieves instructions from the instruction cache, followed by a DECODE stage that prepares them for execution. Subsequently, a series of issue stages-ISS0, ISS1, and ISS2-are responsible for generating the coordinates and memory addresses for the result and input operands. Finally, the ISS3 stage produces the necessary control signals for the Cube Unit, Vector Unit, TR, and the intra-lane data paths, completing the instruction dispatch process.

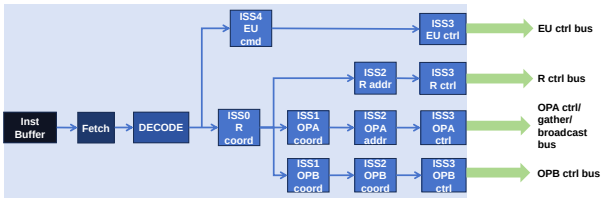


Figure 17: TIU Pipeline.

4 Toolchain

The OASIS's toolchain is an improved version of TPU-MLIR project [20, 24]. The updated toolchain flow is illustrated in the Fig. 18, with the primary improvements highlighted in red and some unchanged parts omitted.

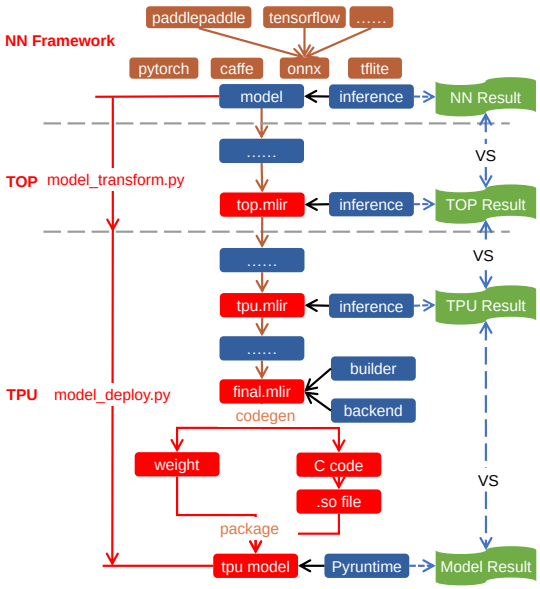


Figure 18: Toolchain.

The toolchain provides broad compatibility with popular AI frameworks. It natively supports models of three frameworks: Pytorch, Caffe, and Tensorflow-lite. Models of other frameworks such as PaddlePaddle and Tensorflow can be converted to the Open Neural Network Exchange (ONNX) format. Our frontend parser ingests the graph and converts it into a high-level representation using MLIR's `TOP` dialect. Subsequently, through multiple stages of parsing and conversion, with the dialect as the output of each stage, the model is converted into machine code that ultimately runs on the chip.

A series of compiler passes are applied to optimize the computation graph. These include standard optimizations like constant folding and dead code elimination, as well as AI-specific passes like operator fusion. For example, a convolution layer followed by a Rectified Linear Unit (ReLU) activation and batch normalization can be fused into a single, highly efficient kernel for the RTPU, reducing memory traffic and launch overhead.

Our approach to quantization is defined by a tight hardware-software co-design. On the hardware level, the OASIS RTPU is a highly programmable architecture with explicit, instruction-level support for INT4/INT8/FP16/BF16 quantization operations, more complete and flexible than the well-known AI accelerator NVDLA. A key differentiator is our native de-quant and re-quant instruction (dq0, dq1, dq2, rq1, rq2), which provides hardware acceleration for the 4-bit weight dequantization critical for modern LLMs-a capability the NVDLA architecture lacks. On the compiler side, our toolchain implements a user-friendly Post-Training Quantization (PTQ) flow that is philosophically similar to TensorRT's. This automated process involves a calibration step, where the tool analyzes the distribution of weights and activations on a small, representative dataset to calculate the optimal scaling factors that minimize accuracy loss. However, our toolchain's novelty lies in its ability to target our flexible hardware, intelligently mapping parts of the quantization and dequantization process directly to our custom instructions. This co-design allows OASIS to efficiently support not only the standard INT8 CNN workloads, but also the mixed-precision and low-bit-width requirements of emerging models, enabling developers to leverage the performance benefits of low-precision inference with minimal effort.

The novelty of our toolchain lies in a sophisticated, multi-stage transformation process within the MLIR framework that intelligently bridges the gap between a high-level model and our specialized hardware. Our approach begins with hardware-aware operator fusion, where common sequences like `Conv+BN+ReLU` are not just combined, but are mapped to custom fused-ops that are co-designed with the RTPU's efficient execution capabilities. This is followed by a global memory planning and layout transformation pass, which strategically schedules DMA transfers and reorganizes data layouts to maximize on-chip memory reuse and minimize DRAM traffic. Finally, a key innovation is our backend's lowering to human-readable C-code with intrinsics, rather than a proprietary binary. This transform makes the compiler's output transparent, debuggable, and allows expert developers to easily integrate hand-optimized code, offering an advantage in programmability and extensibility over traditional "black-box" compilation flows.

5 Evaluation

We present an evaluation of the performance of OASIS against BM1684X [20], and the NVIDIA Jetson AGX Orin. Table 4 summarizes the key characteristics, serving as a basis to compare their ability to handle AI workloads.

Table 4: Physical Implementation Details of Evaluated Platforms.

Processor	Node (nm)	Freq. (GHz)	Voltage (V)	TOPS (INT8)	DRAM (GB/s)	TDP (W)
OASIS	12	1.0	0.8	2	204.8	50
BM1684X	12	1.0	~0.8	32	68.3	25
Jetson AGX Orin	8	1.3	~0.75	275	204.8	60

5.1 Architectural Insight via Roofline Analysis

To move beyond relative comparisons and understand the absolute efficiency of our architecture against its theoretical limits, we construct a Roofline model for the OASIS processor, plotting its theoretical peak 16 TFLOPS FP16 compute performance against its 204.8 GB/s DRAM bandwidth. Our analysis of key benchmarks on this model (Fig. 19) reveals that OASIS achieves a strong balance across the entire arithmetic intensity spectrum. For compute-bound workloads like ResNet-50 in 16 batch scenario with 2500 FLOPS/BYTE, our measured performance reaches of the 58.7% of theoretical peak, validating the high efficiency of our hardware data path and on-chip memory hierarchy. For memory-bound tasks like the decode phase of Llama2-7B in W4A16 scenario with 4 FLOPS/BYTE, our measured performance reaches of the 61.5% of theoretical peak, demonstrating an efficient memory subsystem. Furthermore, for balanced workloads like YOLOv8-s (~26625 FLOPS/BYTE) that lie near the model's "ridge point", our architecture effectively utilizes both compute and memory resources. This analysis provides a key insight: OASIS demonstrates high architectural efficiency across the entire workload spectrum. Its performance for compute-bound and memory-bound/balanced workloads is well-aligned with the system's peak compute capabilities and available memory bandwidth, respectively, indicating high overall hardware utilization. This quantitatively demonstrates the success and versatility of our balanced design choices.

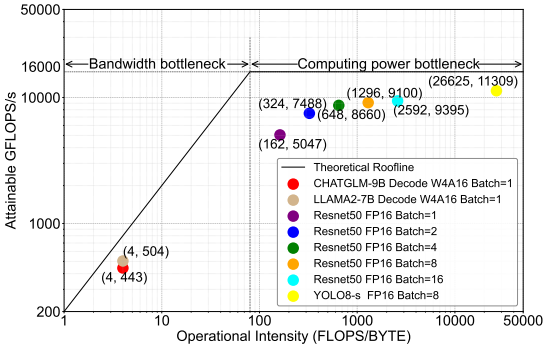


Figure 19: Roofline Model for the OASIS AI Processor.

5.2 Performance on Diverse AI Workloads

OASIS supports a wide variety of AI workloads, we focus on three distinct and representative benchmarks to stress different aspects of our architecture, from compute-intensive operations to memory-bandwidth-sensitive tasks. We consider four key metrics: throughput, throughput relative to computing power, throughput relative to memory bandwidth, and computing power relative to power. To create a more direct "apples-to-apples" comparison against the Jetson AGX Orin's 8nm process, we analytically model a hypothetical "OASIS-8nm" processor by normalizing our measured 12nm results. We project that an 8nm version of OASIS would benefit from the node shrink, resulting in an estimated 20% reduction in power while delivering the same throughput. This model allows for a direct comparison of the architectural energy efficiency by neutralizing the process technology advantage.

5.2.1 Image Classification (ResNet-50). The results for ResNet-50, a compute-intensive CNN, are shown in Fig. 20. While the AGX Orin's raw throughput is higher due to its massive peak TOPS, OASIS achieves a 292% higher throughput-to-computing-power ratio. This indicates a more efficient use of its available hardware resources.

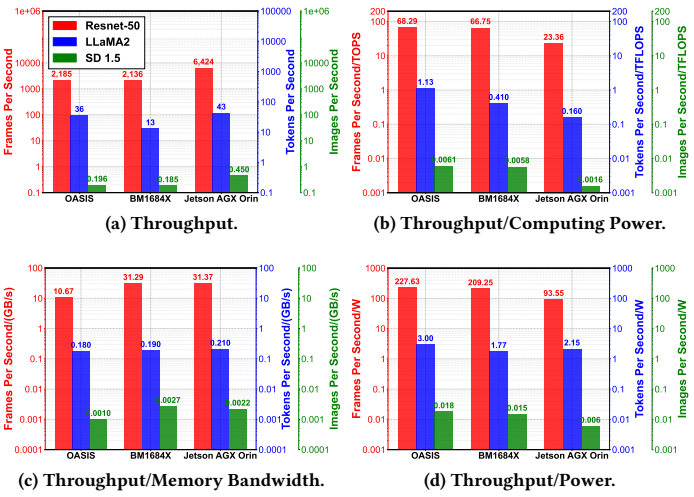
This superior efficiency stems directly from our hardware-accelerated data path. For compute-bound CNNs like ResNet-50, performance is dominated by the efficiency of convolutional operations. The RTPU's specialized data-gathering system (our modular crossbar and ring bus) performs the `im2col` data rearrangement in hardware with minimal overhead. In contrast, a GPU-based platform like the Orin must execute this task in software via general-purpose CUDA kernels, incurring instruction and data movement overhead which leads to lower utilization of its tensor cores. Furthermore, the comparable performance between OASIS and the BM1684X confirms that for this workload, DRAM bandwidth is not the primary bottleneck; rather, the effective utilization of on-chip compute is the key determining factor.

5.2.2 Language Generation (Llama2-7B). We evaluate single-batch Llama2-7B inference using 4-bit weight and 16-bit activation quantization, a common edge computing scenario. As shown in Fig. 20, despite the AGX Orin possessing 859% more theoretical computing power, its throughput is only ~20% higher than OASIS. This results in OASIS achieving a 719% higher throughput-to-computing-power ratio.

This discrepancy is explained by the nature of the LLM decode phase. The use of a KV cache makes this operation heavily memory-bound, dominated by matrix-vector multiplications which have low arithmetic intensity. The massive parallel compute units of the AGX Orin are severely underutilized as they are constantly starved for data. OASIS, with its balanced architecture, performs exceptionally well in this regime. Furthermore, OASIS's performance is approximately three times that of the BM1684X. This gain is attributed to two key advantages: a 3× higher DRAM bandwidth (204.8 GB/s vs. 68.3 GB/s), which is critical for this memory-bound task, and the inclusion of our native `t.dequant.s4.bf16` hardware instruction for efficient 4-bit weight dequantization, reducing what would otherwise be a software overhead.

5.2.3 Image Generation (Stable Diffusion 1.5). To further verify its versatility, we test the Stable Diffusion 1.5 model (512×512 resolution), a latency-sensitive creative generation task. This workload is particularly demanding as it involves dozens of iterative denoising steps through a large U-Net architecture. As shown in Fig. 20, OASIS demonstrates a higher generation speed and 275% better compute power efficiency (images/TFLOPS) compared to the Jetson Orin.

This efficiency gain stems from two key architectural advantages when handling U-Net style workloads. First, the U-Net's structure, with its numerous skip-connections, creates a high demand for on-chip memory to hold intermediate feature maps. The RTPU's large, 4MB compiler-managed LMEM is superior to a general-purpose cache in this scenario, as our toolchain can explicitly orchestrate data placement to maximize reuse and minimize off-chip DRAM traffic. Second, each denoising step involves a full pass through the network's convolutional layers. The RTPU's hardware-accelerated



(a) Throughput. (b) Throughput/Computing Power.

(c) Throughput/Memory Bandwidth. (d) Throughput/Power.

Figure 20: Performance on Three benchmarks.

`im2col` data path efficiently handles these operations with minimal overhead in every iteration, whereas a GPU-based approach incurs repeated software overhead. This combination of superior on-chip memory management and efficient data-path execution allows OASIS to maintain high utilization and efficiency throughout the entire iterative generation process.

To demonstrate the generality of OASIS, we benchmark its performance across a diverse suite of 11 representative models, with results presented in Table 5. Our evaluation confirms high efficiency on not only foundational models like MobileNetV3 and Vision Transformers, but also on large-scale generative AI.

Table 5: Model Performance Comparison

Model Name	Version	Param Type	Param Size (MB)	Latency (ms)
ChatGLM	4-9B	INT4	4380	Decoder: 39.46 Prefill: 1431.69
Qwen	2-7B	INT4	3508	Decoder: 49.67 Prefill: 1149.23
SAM encoder	ViT H/16	FP16	334	166.76
Grounding-DINO	Swin-T	INT8	532	115.07
yolo	v8-s	INT8	12	3.97 (batch size=1) 7.90 (batch size=4) 15.07 (batch size=8) 29.30 (batch size=16)
yolo	v8-l	INT8	45	11.30 (batch size=1) 32.46 (batch size=4) 63.82 (batch size=8) 126.70 (batch size=16)
mobilenet	v3-large	INT8	6	2.30
Clip		INT8	106	5.66
ViT	Base	INT8	83	5.27
ViT	Large	INT8	393	51.92

5.3 Validation of Scalability and Extensibility

A core contribution of OASIS is its scalable, multi-core architecture. The following experiments are designed to quantitatively verify these claims of extensibility.

5.3.1 Multi-Core Scaling. To test the efficiency of our architecture and novel message-based synchronization mechanism, we perform a scaling experiment using the ResNet-50 benchmark. The workload is parallelized by our toolchain to run across 1, 2, and 4 RTPU cores.

The experiment shows that performance scales near-linearly with the number of cores. We achieve a speedup of 1.92× on 2 cores and 3.75× on 4 cores, reaching 93.8% of ideal linear speedup. This provides powerful, quantitative evidence that our low-overhead synchronization scheme and memory subsystem are effective, avoiding the bottlenecks that typically limit multi-core accelerator performance.

5.3.2 Multi-Model Concurrency. A critical use case for terminal devices is running multiple AI applications concurrently. We validate this capability by launching four independent streams of the ResNet-50 model, with the OASIS runtime environment assigning each stream to a dedicated RTPU core.

The system sustains an aggregate throughput of 2072 FPS. The average per-stream throughput is 518 FPS, which is over 90% of the single-core, single-stream performance. This minimal degradation demonstrates excellent performance isolation between the RTPU cores and confirms that the DRAM memory controller provide sufficient bandwidth to service all four cores simultaneously without contention. This result validates OASIS's suitability for complex, multi-tasking AI workloads at the terminal.

6 Related Work

Recent years have seen a surge of AI processors based on the RISC-V architecture, demonstrating its tremendous potential [6, 10, 32, 34, 50, 57, 61–65]. Early RISC-V-based AI processors, such as the SiFive Intelligence X280 [58], tightly integrate a vector processing unit that shares the main core's LSU and memory hierarchy. While this simplifies the design, however, it also introduces a bottleneck: the vector/AI units must compete with scalar cores for highly limited memory bandwidth, leading to stalls and underutilization. Meanwhile, RVV exhibits relatively low efficiency for AI models where convolutions and matrix multiplications account for the vast majority of computations. More recent designs have introduced matrix multiplication extension instructions [10, 26, 49]. For example, Tesla's DOJO D1 chip [10] expands the RISC-V ISA with matrix multiplication instructions. While highly effective for GEMM-based workloads like Transformers, this approach still lacks native support for the sliding-window data patterns of convolution. In addition, the architecture of the shared memory access interface limits the memory access bandwidth, which ultimately limits the computing power ceiling of a single core. Although using Network on Chip (NoC) to connect multiple cores can improve computing power, the low computing power of a single core leads to more nodes in the NOC, which increases the proportion of communication overhead and causes overall performance to decline. In contrast, OASIS architecturally solves the Load & Store bandwidth bottleneck. Its TPU features a completely independent LSU and SMMU, a crucial decoupling that allows for maximum bandwidth tensor streaming from DRAM and LMEM without contention from the SVPU. This efficient data supply feeds a compute architecture governed by a Tensor Extension ISA. The ISA provides first-class hardware support for both convolution and highly-optimized matrix multiplication. This dual capability ensures high efficiency on both CNNs and Transformers, providing the multimodal versatility that GEMM-only or vector-only approaches lack.

Another phenomenal product outside of RISC-V based AI processors is the Groq TSP, which utilizes a deterministic, compiler-scheduled dataflow model that entirely eliminates external DRAM for computation by relying on a massive on-chip SRAM (over 230MB) [1]. This architectural choice is key to its exceptionally high performance on centain models, but it comes at an area and cost overhead. More importantly, when the model size exceeds the on-chip SRAM capacity of a single chip, multiple chips need to form a cluster, making this architecture choice unsuitable for terminal application scenarios. OASIS leverages the highly promising RISC-V ecosystem, adopting an AI-specific processor with tensor extension instruction support. It features independent LSU and SMMU for efficient memory access, striking a balance between on-chip memory and high-bandwidth external DRAM, while focusing on meeting the demands of running AI models on a single chip. The divergence between the two is so fundamental-with one system balancing on-chip and off-chip memory while the other relies exclusively on-chip-that a direct comparison of metrics like power or TFLOPs utilization would be misleading, as they do not represent the same system-level trade-offs.

OASIS's superior performance over the Jetson AGX Orin stems from a triad of architectural advantages [48]. It replaces the GPU's power-hungry warp scheduler with a streamlined, coarse-grained instruction model. It attacks the dominant cost of data movement by using a large, compiler-managed scratchpad and dedicated hardware for data rearrangement (`im2col`), avoiding the inefficiencies of general-purpose caches. Lastly, its programmability provides a wider efficiency profile than the rigid NVDLA, preventing costly fallbacks to the GPU for unsupported operators. OASIS thus occupies a deliberately chosen, balanced point in the design space: specialized enough to be highly efficient, yet flexible enough for diverse, modern AI workloads.

Compared to prior commercial designs like the Sophgo BM1684X [20], OASIS introduces fundamental architectural and programming model innovations to enhance flexibility and scalability for terminal devices. In sharp contrast to the high-latency coprocessor model of BM1684X, OASIS elevates the AI accelerator to a first-class architectural unit by integrating Tensor instructions into the same unified stream as standard RISC-V instructions, enabling low-latency, fine-grained synergy. This is complemented by a novel toolchain that generates human-readable C code with tensor intrinsics over a proprietary, "black-box" binary, making the platform more transparent, debuggable, and extensible. Finally, where the BM1684X uses a monolithic TPU, OASIS's primary architectural contribution is its scalable, 4-core RTPU design, built for multi-model concurrency via a NOC and a low-overhead synchronization mechanism—an essential capability for modern, multi-tasking terminal devices not addressed by prior monolithic designs.

7 Conclusion and Lessons Learned

This paper introduces OASIS, a commercial terminal AI processor that served as a vehicle for investigating and validating a new set of architectural principles for high-performance, flexible AI acceleration on RISC-V. Through its design, fabrication, and evaluation, we have derived and empirically validated several key lessons for the community. Our work proves that hardware acceleration for data rearrangement (e.g., `im2col`) is not an optimization but a necessity for achieving high compute utilization in workloads like CNNs. We quantitatively demonstrate that accelerator memory subsystems must be decoupled to prevent crippling performance degradation in realistic, multi-tasking scenarios. Finally, we show that a specialized, low-overhead synchronization mechanism is a far more efficient and scalable solution for multi-core accelerators than general-purpose, cache-coherent methods. By establishing these evidence-backed principles, this work provides a new, validated blueprint for the next generation of open-standard AI processors and will guide future efforts in building a truly high-performance, scalable, and versatile AI computing ecosystem.

Acknowledgments

This work was supported by the National Key Research and Development Program of China under Grant No. 2024YFB2906602. We thank the anonymous reviewers for their insightful and constructive feedback.

References

[1] Dennis Abts, Jonathan Ross, Jonathan Sparling, Mark Wong-VanHaren, Max Baker, Tom Hawkins, Andrew Bell, John Thompson, Temesghen Kahsai, Garrin Kimmell, Jennifer Hwang, Rebekah Leslie-Hurd, Michael Bye, E.R. Creswick, Matthew Boyd, Mahitha Venigalla, Evan Laforge, Jon Purdy, Purushotham Kamath, Dinesh Maheshwari, Michael Beidler, Geert Rosseel, Omar Ahmad, Gleb Gagarin, Richard Czekalski, Ashay Rane, Sahil Parmar, Jeff Werner, Jim Sproch, Adrian Macias, and Brian Kurtz. 2020. Think Fast: A Tensor Streaming Processor (TSP) for Accelerating Deep Learning Workloads. In *Proceedings of ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 145–158.

[2] Johnathan Alsop, Matthew D. Sinclair, Rakesh Komuravelli, and Sarita V. Adve. 2016. GSI: A GPU Stall Inspector to Characterize the Sources of Memory Stalls for Tightly Coupled GPUs. In *Proceedings of IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 172–182.

[3] Agathe Archet, Nicolas Gac, François Oriol, and Nicolas Ventroux. 2023. Embedded AI Performances of NVIDIA's Jetson Orin SoC Series. In *17éme Colloque National du GDR SOC2*.

[4] Robert Beachler and Martin Snelgrove. 2022. Untether AI: Boqueria. In *Proceedings of IEEE Hot Chips 34 Symposium (HCS)*. 1–19.

[5] Vincent Camus, Linyan Mei, Christian Enz, and Marian Verhelst. 2019. Review and Benchmarking of Precision-Scalable Multiply-Accumulate Unit Architectures for Embedded Neural-Network Processing. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 4 (2019), 697–711.

[6] Bill Chang, Rajiv Kurian, Doug Williams, and Eric Quinnell. 2022. DOJO: Super-Compute System Scaling for ML Training. In *Proceedings of IEEE Hot Chips 34 Symposium (HCS)*. 1–45.

[7] Yunji Chen, Tianshi Chen, Zhiwei Xu, Ninghui Sun, and Olivier Temam. 2016. DianNao Family: Energy-Efficient Hardware Accelerators for Machine Learning. *Commun. ACM* 59, 11 (2016), 105–112.

[8] Yiran Chen, Yuan Xie, Linghao Song, Fan Chen, and Tianqi Tang. 2020. A Survey of Accelerator Architectures for Deep Neural Networks. *Engineering* 6, 3 (2020), 264–274.

[9] Yuan-Hu Cheng, Li-Bo Huang, Yi-Jun Cui, Sheng Ma, Yong-Wen Wang, and Bing-Cai Sui. 2022. RV16: An Ultra-Low-Cost Embedded RISC-V Processor Core. *Journal of Computer Science and Technology* 37, 6 (2022), 1307–1319.

[10] Shu-Rong Chun, Tin-Hao Kuo, Hao-Yi Tsai, Chung-Shi Liu, Chuei-Tang Wang, Jeng-Shien Hsieh, Tsung-Shu Lin, Terry Ku, and Douglas Yu. 2020. InFO_SoW (System-on-Wafer) for High Performance Computing. In *Proceedings of IEEE 70th Electronic Components and Technology Conference (ECTC)*. 1–6.

[11] Pudi Dhilleswararao, Srinivas Boppu, M. Sabarimalai Manikandan, and Linga Reddy Cenkeramaddi. 2022. Efficient Hardware Architectures for Accelerating Deep Neural Networks: Survey. *IEEE Access* 10 (2022), 131788–131828.

[12] Dave Ditzel, Roger Espasa, Nivard Aymerich, Allen Baum, Tom Berg, Jim Burr, Eric Hao, Jayesh Iyer, Miquel Izquierdo, Shankar Jayaratnam, Darren Jones, Chris Klingner, Jin Kim, Stephen Lee, Marc Lupon, Grigorios Magklis, Bojan Maric, Rajib Nath, Mike Neilly, Duane Northcutt, Bill Orner, Jose Renau, Gerard Reves, Xavier Reves, Tom Riordan, Pedro Sanchez, Sri Samudrala, Guillem Sole, Raymond Tang, Tommy Thorn, Francisco Torres, Sebastia Tortella, and Daniel Yau. 2021. Accelerating ML Recommendation With Over 1,000 RISC-V/Tensor Processors on Esperanto's ET-SoC-1 Chip. In *Proceedings of IEEE Hot Chips 33 Symposium (HCS)*. 1–23.

[13] David R. Ditzel and the Esperanto team. 2022. Accelerating ML Recommendation With Over 1,000 RISC-V/Tensor Processors on Esperanto's ET-SoC-1 Chip. *IEEE Micro* 42, 3 (2022), 31–38.

[14] Jaeyoung Do, Victor C. Ferreira, Hossein Bobarshad, Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Diego Souza, Brunno F. Goldstein, Leandro Santiago, Min Soo Kim, Priscila M. V. Lima, Felipe M. G. França, and Vladimir Alves. 2020. Cost-effective, Energy-Efficient, and Scalable Storage Computing for Large-Scale AI Applications. *ACM Transactions on Storage* 16, 4 (2020), 1–37.

[15] Zidong Du, Qi Guo, Yongwei Zhao, Tian Zhi, Yunji Chen, and Zhiwei Xu. 2020. Self-Aware Neural Network Systems: A Survey and New Perspective. *Proc. IEEE* 108, 7 (2020), 1047–1067.

[16] Saumay Dublish, Vijay Nagarajan, and Nigel Topham. 2019. Poise: Balancing Thread-Level Parallelism and Memory System Performance in GPUs Using Machine Learning. In *Proceedings of IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 492–505.

[17] Ahmed ElTantawy, Jessica Wenjie Ma, Mike O'Connor, and Tor M. Aamodt. 2014. A Scalable Multi-path Microarchitecture for Efficient GPU Control Flow. In *Proceedings of IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. 248–259.

[18] Babak Falsafi. 2023. What's Missing in Agile Hardware Design? Verification! *Journal of Computer Science and Technology* 38, 4 (2023), 735–736.

[19] Farzad Farshchi, Qijing Huang, and Heechul Yun. 2019. Integrating NVIDIA Deep Learning Accelerator (NVDLA) with RISC-V SoC on FireSim. In *Proceedings of the 2nd Workshop on Energy Efficient Machine Learning and Cognitive Computing for Embedded Applications (EMC2)*. 21–25.

[20] Peng Gao, Yang Liu, Jun Wang, Wanlin Cai, Guangchong Shen, Zonghui Hong, Jiali Qu, and Ning Wang. 2024. SOPHGO BM1684X: A Commercial High Performance Terminal AI Processor with Large Model Support. In *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1413–1428.

[21] Xiang Gong, Zhongliang Chen, Amir Kavyan Ziabari, Rafael Ubal, and David Kaeli. 2017. TwinKernels: An Execution Model to Improve GPU Hardware Scheduling at Compile Time. In *Proceedings of IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 39–49.

[22] Xun Gong, Xiang Gong, Leiming Yu, and David Kaeli. 2019. HAWS: Accelerating GPU Wavefront Execution Through Selective Out-of-Order Execution. *ACM Transactions on Architecture and Code Optimization* 16, 2 (2019), 1–22.

[23] John L. Hennessy and David A. Patterson. 2019. A New Golden Age for Computer Architecture. *Commun. ACM* 62, 2 (2019), 48–60.

[24] Pengchao Hu, Man Lu, Lei Wang, and Guoyue Jiang. 2023. TPU-MLIR: A Compiler For TPU Using MLIR. arXiv:2210.15016 [cs.PL] https://arxiv.org/abs/2210.15016

[25] Angel Hsing-Chi Hwang, John Oliver Siy, Renee Shelby, and Alison Lentz. 2024. In Whose Voice?: Examining AI Agent Representation of People in Social Interaction through Generative Speech. In *Proceedings of the 2024 ACM Designing Interactive Systems Conference (DIS)*. 224–245.

[26] Drago Ignjatović, Daniel W. Bailey, and Ljubisa Bajić. 2022. The Wormhole AI Training Processor. In *Proceedings of IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. 356–358.

[27] Summaira Jabeen, Xi Li, Muhammad Shoib Amin, Omar Bourahla, Songyuan Li, and Abdul Jabbar. 2023. A Review on Methods and Applications in Multimodal Deep Learning. *ACM Transactions on Multimedia Computing, Communications, and Applications* 19, 2s (2023), 1–41.

[28] Zhe Jia, Marco Maggioni, Jeffrey Smith, and Daniele Paolo Scarpazza. 2019. Dissecting the NVidia Turing T4 GPU via Microbenchmarking. arXiv:1903.07486 [cs.DC] https://arxiv.org/abs/1903.07486

[29] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. arXiv:2304.01433 [cs.AR] https://arxiv.org/abs/2304.01433

[30] Norman P. Jouppi, Cliff Young, Nishant Patil, and David Patterson. 2018. A Domain-Specific Architecture for Deep Neural Networks. *Commun. ACM* 61, 9 (2018), 50–59.

[31] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati,

William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. arXiv:1704.04760 [cs.AR] https://arxiv.org/abs/1704.04760

[32] Dongyun Kam, Jung Gyu Min, Jongho Yoon, Sunmean Kim, Seokhyeong Kang, and Youngjoo Lee. 2022. Design and Evaluation Frameworks for Advanced RISC-based Ternary Processor. In *Proceedings of the 2022 Conference & Exhibition on Design, Automation & Test in Europe (DATE '22)*. 1077–1082.

[33] Aajna Karki, Chethan Palangotu Keshava, Spoorthi Mysore Shivakumar, Joshua Skow, Goutam Madhukeshwar Hegde, and Hyeran Jeon. 2019. Tango: A Deep Neural Network Benchmark Suite for Various Accelerators. In *Proceedings of IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 137–138.

[34] Ajay Kumar M, Vineet Kumar, Deepu John, and Shreejith Shanker. 2024. Implementation and Analysis of Custom Instructions on RISC-V for Edge-AI Applications. In *Proceedings of the 14th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies (HEART)*. 126–129.

[35] NVIDIA Jetson AI Lab. 2024. Generative AI at the Edge. https://www.jetson-ai-lab.com/

[36] Hamed F. Langroudi, Tej Pandit, Mark Indovina, and Dhireesha Kudithipudi. 2019. Digital Neuromorphic Chips for Deep Learning Inference: A Comprehensive Study. In *Applications of Machine Learning*, Vol. 11139. 111390C.

[37] Cecilia Latotzke and Tobias Gemmeke. 2021. Efficiency Versus Accuracy: A Review of Design Techniques for DNN Hardware Accelerators. *IEEE Access* 9 (2021), 9785–9799.

[38] Dylan Lee, Shaoyuan Xie, Shagoto Rahman, Kenneth Pat, David Lee, and Qi Alfred Chen. 2024. "Prompter Says": A Linguistic Approach to Understanding and Detecting Jailbreak Attacks Against Large-Language Models. In *Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis*. 77–87.

[39] Ang Li, Shuaiwen Leon Song, Jieyang Chen, Jiajia Li, Xu Liu, Nathan R. Tallent, and Kevin J. Barker. 2020. Evaluating Modern GPU Interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect. *IEEE Transactions on Parallel and Distributed Systems* 31, 1 (2020), 94–110.

[40] Ruoshi Li, Ping Peng, Zhiyuan Shao, Hai Jin, and Ran Zheng. 2023. Evaluating RISC-V Vector Instruction Set Architecture Extension with Computer Vision Workloads. *Journal of Computer Science and Technology* 38 (2023), 807–820.

[41] Raju Machupalli, Masum Hossain, and Mrinal Mandal. 2022. Review of ASIC Accelerators for Deep Neural Network. *Microprocessors and Microsystems* 89 (2022), 104441.

[42] Vincenzo Maisto and Alessandro Cilardo. 2022. A Pluggable Vector Unit for RISC-V Vector Extension. In *Proceedings of Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1143–1148.

[43] Diksha Moolchandani, Anshul Kumar, and Smruti R. Sarangi. 2021. Accelerating CNN Inference on ASICs: A Survey. *Journal of Systems Architecture* 113 (2021), 101887.

[44] Pascal Nasahl, Robert Schilling, Mario Werner, and Stefan Mangard. 2021. HECTOR-V: A Heterogeneous CPU Architecture for a Secure RISC-V Execution Environment. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security (ASIA CCS '21)*. 187–199.

[45] Nvidia. 2015. NVIDIA Multi-Process Service. https://docs.nvidia.com/deploy/mps/index.html

[46] Nvidia. 2017. Nvidia Tesla V100 GPU Architecture. https://images.nvidia.cn/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

[47] NVIDIA. 2022. NVIDIA Jetson AGX Orin Series. https://www.nvidia.cn/content/dam/en-zz/Solutions/gtcf21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf

[48] NVIDIA. 2024. NVIDIA Jetson Comparison. https://www.seeedstudio.com/blog/nvidia-jetson-comparison-nano-tx2-nx-xavier-nx-agx-orin/

[49] David Patterson. 2018. 50 Years of Computer Architecture: From the Mainframe CPU to the Domain-Specific TPU and the Open RISC-V Instruction Set. In *Proceedings of IEEE International Solid-State Circuits Conference (ISSCC)*. 27–31.

[50] Eric Quinnell. 2024. Tesla Transport Protocol Over Ethernet (TTPoE): A New Lossy, Exa-Scale Fabric for the Dojo AI Supercomputer. In *Proceedings of IEEE Hot Chips 36 Symposium (HCS)*. 1–23.

[51] Albert Reuther, Peter Michaleas, Michael Jones, Vijay Gadepally, Siddharth Samsi, and Jeremy Kepner. 2019. Survey and Benchmarking of Machine Learning Accelerators. In *Proceedings of IEEE High Performance Extreme Computing Conference (HPEC)*. 1–9.

[52] Albert Reuther, Peter Michaleas, Michael Jones, Vijay Gadepally, Siddharth Samsi, and Jeremy Kepner. 2020. Survey of Machine Learning Accelerators. In *Proceedings of IEEE High Performance Extreme Computing Conference (HPEC)*. 1–12.

[53] Albert Reuther, Peter Michaleas, Michael Jones, Vijay Gadepally, Siddharth Samsi, and Jeremy Kepner. 2021. AI Accelerator Survey and Trends. In *Proceedings of IEEE High Performance Extreme Computing Conference (HPEC)*. 1–9.

[54] Albert Reuther, Peter Michaleas, Michael Jones, Vijay Gadepally, Siddharth Samsi, and Jeremy Kepner. 2022. AI and ML Accelerator Survey and Trends. In *Proceedings of IEEE High Performance Extreme Computing Conference (HPEC)*. 1–10.

[55] Albert Reuther, Peter Michaleas, Michael Jones, Vijay Gadepally, Siddharth Samsi, and Jeremy Kepner. 2023. Lincoln AI Computing Survey (LAICS) Update. arXiv:2310.09145 [cs.AI] https://arxiv.org/abs/2310.09145

[56] Génération ROBOTS. 2022. Our NVIDIA Jetson Orin and NVIDIA Jetson Xavier Comparison. https://www.generationrobots.com/blog/en/our-nvidia-jetson-orin-and-nvidia-jetson-xavier-comparison/

[57] Bruno Sá, Francisco Marques, Manuel Rodriguez, José Martins, and Sandro Pinto. 2023. Holistic RISC-V Virtualization: CVA6-Based SoC. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*. 389–390.

[58] SiFive. 2022. SiFive Intelligence & VCIX. https://www.kisacoresearch.com/sites/default/files/presentations/1pdf._sifive_ai_presentation-kasanovic.pdf

[59] Martin Snelgrove and Robert Beachler. 2023. SpeedAI240: A 2-Petaflop, 30-Teraflops/W At-Memory Inference Acceleration Device With 1456 RISC-V Cores. *IEEE Micro* 43, 3 (2023), 58–63.

[60] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2017. Efficient Processing of Deep Neural Networks: A Tutorial and Survey. *Proc. IEEE* 105, 12 (2017), 2295–2329.

[61] Thomas Szymkowiak, Endrit Isufi, and Markku-Juhani Saarinen. 2024. Poster: Marian: An Open Source RISC-V Processor With Zvk Vector Cryptography Extensions. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. 4931–4933.

[62] Emil Talpes, Debjit Das Sarma, Doug Williams, Sahil Arora, Thomas Kunjan, Benjamin Floering, Ankit Jalote, Christopher Hsiong, Chandrasekhar Poorna, Vaidehi Samant, John Sicilia, Anantha Kumar Nivarti, Raghuvir Ramachandran, Tim Fischer, Ben Herzberg, Bill McGee, Ganesh Venkataramanan, and Pete Banon. 2023. The Microarchitecture of DOJO, Tesla's Exa-Scale Computer. *IEEE Micro* 43, 3 (2023), 31–39.

[63] Emil Talpes, Douglas Williams, and Debjit Das Sarma. 2022. DOJO: The Microarchitecture of Tesla's Exa-Scale Computer. In *Proceedings of IEEE Hot Chips 34 Symposium (HCS)*. 1–28.

[64] Hidetaro Tanaka, Tomoaki Tanaka, Ryosuke Higashi, Tsutomu Sekibe, Shuichi Takada, and Hironori Nakajo. 2022. Implementation of A RISC-V SMT Core in an AI Processor. In *Proceedings of the 11th International Symposium on Information and Communication Technology (SoICT)*. 15–22.

[65] Liang-Ni Wang, Ju-Hung Li, Chi-Bang Kuan, and Yi-Chiao Su. 2024. Support Post Quantum Cryptography with SIMD Everywhere on RISC-V Architectures. In *Workshop Proceedings of the 53rd International Conference on Parallel Processing (ICPP Workshops '24)*. 23–32.

[66] Yi-Nan Xu, Zi-Hao Yu, Kai-Fan Wang, Hua-Qiang Wang, Jia-Wei Lin, Yue Jin, Lin-Juan Zhang, Zi-Fei Zhang, Dan Tang, Sa Wang, Kan Shi, Ning-Hui Sun, and Yun-Gang Bao. 2023. Functional Verification for Agile Processor Development: A Case for Workflow Integration. *Journal of Computer Science and Technology* 38, 4 (2023), 737–753.

[67] Jing Zhang and Dacheng Tao. 2021. Empowering Things With Intelligence: A Survey of the Progress, Challenges, and Opportunities in Artificial Intelligence of Things. *IEEE Internet of Things Journal* 8, 10 (2021), 7789–7817.

[68] Zhongyi Zhou, Jing Jin, Vrushank Phadnis, Xiuxiu Yuan, Jun Jiang, Xun Qian, Jingtao Zhou, Yiyi Huang, Zheng Xu, Yinda Zhang, Kristen Wright, Jason Mayes, Mark Sherwood, Johnny Lee, Alex Olwal, David Kim, Ram Iyengar, Na Li, and Ruofei Du. 2024. Experiencing InstructPipe: Building Multi-modal AI Pipelines via Prompting LLMs and Visual Programming. In *Proceedings of Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–5.

A RISC-V Tensor Extension Instruction Set

The tensor extension instructions introduce a set of specialized instructions to accelerate tensor-based computations, commonly used in AI workloads. These instructions expand the basic RISC-V instruction set by incorporating 32-bit and 64-bit formats that map to the Custom-2 instruction format, enabling efficient handling of tensor operations. The instructions include various types, such as configuration instructions, memory operations, computational instructions, and synchronization primitives. By integrating these tensor-specific operations into the instruction pipeline, performance gains are achieved in AI computation tasks such as matrix multiplication, convolution, and pooling. The structure and encoding of these instructions are carefully designed to balance flexibility with efficiency, ensuring that both standard scalar and vector operations as well as advanced tensor manipulations can be executed with minimal overhead. In the following sections, we detail the instruction formats and functions.

A.0.1 Instruction Formats. The tensor extension instructions encompass both 32-bit and 64-bit formats, which are mapped to the Custom-2 instruction format in the RISC-V architecture, as shown in Fig. 21. These specialized formats enable efficient execution of tensor operations by extending the basic instruction set. The 32-bit format is used for simpler tensor operations, while the 64-bit format supports more complex operations, allowing for higher precision and larger data throughput. Using the Custom-2 format, these instructions are seamlessly integrated into the RISC-V pipeline, maintaining compatibility with existing scalar and vector operations while enabling enhanced tensor processing capabilities. This design facilitates the efficient handling of AI-specific computations, such as matrix multiplication, convolution, and pooling, directly within the instruction stream.

A.0.2 CSR, CR, TVR, and GVR Configuration Instructions. These instructions are responsible for configuring CSRs, CRs, TVRs, and GVRs, as shown in Figs. 22-23. Due to limitations in the bit width of the extended instructions, many details required by LS, MOVE, and computation instructions are pre-loaded into these registers via configuration instructions. During instruction decoding, the IFE reads the information stored in these registers and encapsulates it along with the decoded instruction. The encapsulated instruction is then sent to the execution unit for processing. This approach ensures that the necessary configuration data are readily available for tensor operations, optimizing the overall performance of the system by minimizing delays associated with instruction execution and register setup.

A.0.3 MOVE Instructions. The MOVE instructions facilitate data transfer between the Vector Register File (VRF) and LMEM, as shown in Fig. 24. They support the transfer of one or two sets of VRF data, which can be either contiguous or scattered across LMEM's address space. Due to the inherent difference in the mapping of physical storage between LMEM and TR, the MOVE instructions indirectly enable data exchange between the VRF and TR.

A.0.4 LOAD & STORE Instructions. The LOAD & STORE (LS) instructions are responsible for managing the data transfer between TR and DRAM, as shown in Figs. 25-26. These instructions support efficient tensor data movement with enhanced capabilities, such as performing transpose and broadcasting along the C dimension during data transfer. In addition, they facilitate data replication between TR and GT or between different TR regions, with support for simultaneous transpose, dimension reversal, and C-dimension broadcasting. The instructions also include advanced features such as Mask Select and Non-Zero Index, allowing for selective data retrieval based on specific conditions. Furthermore, they enable Scatter and Gather operations along the H dimension, offering flexible data manipulation.

A.0.5 COMPUTE Instructions. The Compute instructions are designed to perform tensor computations within the TR, as shown in Figs. 27-29. Building upon basic arithmetic operations such as integer and floating-point arithmetic, data-type conversion, and specialized operations, we introduce several AI-specific instructions to accelerate deep learning workloads. These include convolution and CUBE matrix multiplication instructions, which are essential for many AI algorithms, as well as pooling instructions that support Max Pooling, Average Pooling, Region of Interest (RoI) Pooling, and Depth-wise Convolution. Additionally, the Re-Quantization (RQ) and Dequantization (DQ) instructions cater to AI algorithms' precision adjustments, while the fully connected matrix multiplication instructions enable efficient processing of the Full Connect layer in neural networks.

These specialized Compute instructions reduce the instruction density during the execution of the AI model, thus optimizing both the computation and the memory access efficiency. Furthermore, the Compute instructions include powerful data manipulation features for TR registers, such as broadcasting, transposition, Scatter and Gather operations across one or two dimensions, as well as Mask Select and Non-Zero Index functionalities. These features not only improve computation efficiency but also minimize TR register read/write overhead, enabling faster data processing for AI algorithms.

A.0.6 Synchronization Instructions. Synchronization instructions are used to coordinate the execution of tensor extension instructions both within a single RTPU and across multiple RTPUs, as shown in Fig. 30. These instructions ensure that operations are executed in the correct sequence, enabling efficient parallelism and scalability.

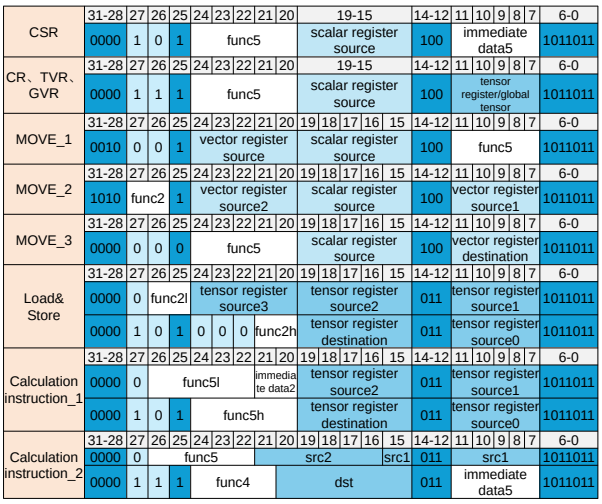


Figure 21: Tensor Extension Instruction Encoding Format.

```
cfg.satu        rs
//Configure whether the results of integer calculation
     instructions are saturated and whether symmetric
     saturation is adopted.
cfg.round_mode rs
// Configure the rounding mode during floating-point
     precision conversion or integer right shift
cfg.rsqrt_iter rs
// Configure the number of Newton iterations for rsqrt
     and fdiv instructions
cfg.quant       ts
// Configure the quantization method, supporting per-
     channel quantization and per-tensor quantization
cfg.pad         rs
// Configure the top, bottom, left, and right padding of
     the feature map for convolution and pooling
     instructions
cfg.insrt       rs
// Configure the interpolation of the feature map or
     kernel for convolution and pooling instructions
cfg.stencil     rs
// Configure the sliding window, stride, whether the
     kernel is rotated, and whether the result is passed
     through ReLU for convolution and pooling
     instructions
cfg.kzp         ts
// Configure the zero point of the kernel for convolution
      and matrix multiplication instructions
cfg.dmaidx      rs
// Configure parameters such as the initial value of the
     index or the default write-back value for
     instructions like ls.hscatter, ls.hgather, and ls.
     nzidx
```

Figure 22: CSR Configuration Instruction.

```
cfgcr           ca, rs
// Configure the data type and constant value of CR
cfgtr           ta, rs
// Configure the data type and offset address of TR
cfgtr.shape     ta, rs
// Configure TR as N, C, H, W
cfgtr.hwstride  ta, rs
// Configure the H stride and W stride of TR
cfgtr.ncstride  ta, rs
// Configure the N stride and C stride of TR
cfggt           ga, rs
// Configure the data type and address of GT
cfggt.shape     ga, rs
// Configure GT as N, C, H, W
cfggt.hwstride  ga, rs
// Configure the H stride and W stride of GT
cfggt.ncstride  ga, rs
// Configure the N stride and C stride of GT
```

Figure 23: CR, TVR, GVR Configuration Instructions.

```
mov.t.v         vs, (rs)
// Load a set of VRF data into LMEM; rs specifies the
     starting relative address in LMEM
mov.dist.v      vs, (rs)
// Distribute a set of VRF data evenly to all LMEMs; rs
     specifies the starting relative address in LMEM
mov.t.vv        vs2, vs1, (rs)
// Load two sets of VRF data into LMEM; rs specifies the
     starting relative address in LMEM
mov.dist.vv     vs2, vs1, (rs)
// Distribute two sets of VRF data evenly to all LMEMs;
     rs specifies the starting relative address in LMEM
mov.v.t         vd, (rs)
// Store data from LMEM into a set of VRF; rs specifies
     the starting relative address in LMEM
mov.v.coll      vd, (rs)
// Collect data from the same relative offset in each
     LMEM and store it into a set of VRF; rs specifies
     the starting relative address in LMEM
```

Figure 24: MOVE Instructions

```
////Data Copy
ls.cp           dst, src
// Copy the source GT or TR to the destination GT or TR
ls.cpbc         dst, src
// Copy the source GT or TR(n, 1, h, w) to the
     destination TR(n, c, h, w)
ls.cpt          dst, src
// Copy the source GT or TR(n, c, h, w) to the
     destination GT or TR(c, n, h, w)
ls.cpr          dst, src, _dim
// Copy the source GT or TR(n, 1, h, w) to the
     destination GT or TR(n, c, h, w) while reversing the
      elements along a specified dimension
```

Figure 25: LOAD & STORE Instructions I.

```
////Tensor Load and Store Instructions
ls.ld        td, gs
// Load the source GT into the destination TR. The shapes
     of the source and destination tensors may differ,
    but the total number of elements must remain the
    same
ls.ldt       td, gs
// Load the source GT(c, n, h, w) into the destination TR
    (n, c, h, w), transposing the N and C dimensions
    during loading
ls.ldbc      td, gs
// Load the source GT(n, 1, h, w) into the destination TR
    (n, c, h, w), broadcasting in the C dimension during
     loading
ls.st        gd, ts
// Store the source TR into the destination GT. The
    shapes of the source and destination tensors may
    differ, but the total number of elements must remain
     the same
ls.stt       gd, ts
// Store the source TR(n, c, h, w) into the destination
    GT(c, n, h, w), transposing the N and C dimensions
    during storing
////Matrix Load and Store Instructions
ls.mld       td, gs
// Load the source GT(1, 1, m, n) into the destination TR
    (m, ROUNDUP(n/CU_NUM), 1, w)
ls.mst       gd, ts
// Store the source TR(m, ROUNDUP(n/CU_NUM), 1, w) into
    the destination GT(1, 1, m, n)
//// Mask & Index Instructions
ls.masksel    dst, src, mask, _bigmask
// Select elements from the source tensor at positions
    where the mask tensor has a value of 1, and output
    them to the destination GT. The mask and source
    tensor can be either GT or TR; the mask tensor is of
     integer type
ls.fmasksel   dst, src, mask, _bigmask
// Select elements from the source tensor at positions
    where the mask tensor has a value of 1, and output
    them to the destination GT. The mask and source
    tensor can be either GT or TR; the mask tensor is of
     floating-point type
ls.nzidx      dst, src
// Treat the source tensor as a one-dimensional vector,
    generate indices of non-zero elements, and output
    them to the destination GT; the elements of the
    source tensor are integers
ls.fnzidx     dst, src
// Treat the source tensor as a one-dimensional vector,
    generate indices of non-zero elements, and output
    them to the destination GT; the elements of the
    source tensor are floating-point numbers
//// Gather & Scatter Instructions
ls.hgather    dst, src, idx, _bigh
// Using elements in the index tensor(1,1,h,1) or (1,c,h
    ,1) as coordinates, gather data from the source
    tensor(1,1,ih,1) or (1,c,ih,1) into the destination
    tensor(1,1,h,1) or (1,c,h,1)
ls.hscatter   dst, src, idx, _bigh, _accu
// Sequentially read data from the source tensor(1,1,ih
    ,1) or (1,c,ih,1), and scatter it to the destination
     tensor(1,1,h,1) or (1,c,h,1) using elements in the
    index tensor(1,1,ih,1) or (1,c,ih,1) as coordinates,
     supporting accumulation into an integer destination
     tensor
ls.fhscatter  dst, src, idx, _bigh, _accu
// Sequentially read data from the source tensor(1,1,ih
    ,1) or (1,c,ih,1), and scatter it to the destination
     tensor(1,1,h,1) or (1,c,h,1) using elements in the
    index tensor(1,1,ih,1) or (1,c,ih,1) as coordinates,
     supporting accumulation into a floating-point
    destination tensor
```

Figure 26: LOAD & STORE Instructions II.

```
////Integer Arithmetic Instructions
add     out, a, b, shift, _satu
// Integer addition, supporting shifting and saturation
sub     out, a, b, shift, _satu
// Integer subtraction, supporting shifting and
    saturation
mul     out, a, b, shift, _satu
// Integer multiplication, supporting shifting and
    saturation
abs     out, a
// Integer absolute value
mac     out, a, b, shift
// Integer multiply-accumulate, supporting shifting and
    saturation
max     out, a, b
// Integer maximum value
min     out, a, b
// Integer minimum value
selgt   out, a, b, c
// If integer a > b, then out = c; otherwise, 0
seleq   out, a, b, c
// If integer a == b, then out = c; otherwise, 0
sellt   out, a, b, c
// If integer a < b, then out = c; otherwise, 0
cmplt   out, a, b, c, d
// Integer: if a < b then c, else d
cmpeq   out, a, b, c, d
// Integer: if a == b then c, else d
cmpgt   out, a, b, c, d
// Integer: if a > b then c, else d
and     out, a, b
// Logical AND
xor     out, a, b
// Logical XOR
or      out, a, b
// Logical OR
not     out, a
// Logical NOT
lshr    out, a, shift
// Logical right shift
ashr    out, a, shift
// Arithmetic right shift
rshr    out, a, shift
// Rotate right shift
clz     out, a
// Count the number of leading zeros
clo     out, a
// Count the number of leading ones
//// Floating-Point Arithmetic Instructions
fadd    out, a, b
// Floating-point addition with optional saturation
fsub    out, a, b
// Floating-point subtraction with optional saturation
fmul    out, a, b
// Floating-point multiplication with optional saturation
fdiv    out, a, b
// Floating-point division with optional saturation
fabs    out, a
// Floating-point absolute value
fmac    out, a, b
// Floating-point multiply-accumulate
fmax    out, a, b
// Floating-point maximum value
fmin    out, a, b
// Floating-point minimum value
fselgt  out, a, b, c
// If floating-point a > b then out = c, otherwise 0
fseleq  out, a, b, c
// If floating-point a == b then out = c, otherwise 0
fsellt  out, a, b, c
// If floating-point a < b then out = c, otherwise 0
fcmplt  out, a, b, c, d
// Floating-point: if a < b then out = c, else out = d
fcmpeq  out, a, b, c, d
// Floating-point: if a == b then out = c, else out = d
fcmpgt  out, a, b, c, d
// Floating-point: if a > b then out = c, else out = d
```

Figure 27: Compute Instructions I.

```
//// Data Type Conversion Instructions
cvt.i2i           out, a
// Integer precision conversion
cvt.i2f           out, a
// Convert integer to float
cvt.f2i           out, a
// Convert float to integer
cvt.f2f           out, a
// Floating-point precision conversion
//// SFU Instructions
sfu.norm          out, a
// Extract exponent part of float and convert to integer
sfu.taylor        out, a, coeff
// Polynomial evaluation
sfu.rsqrt         out, a
// Reciprocal square root
//// Quantization/Dequantization Instructions
dq0               out, a, scale
// The first dequantization: scale and convert to
    floating-point
rq0               out, a, scale
// The first quantization: scale and convert to low-
    precision integer
dq1               out, a, scale
// The second dequantization: scale and convert to high-
    precision integer
rq1               out, a, scale
// The second quantization: scale and convert to low-
    precision integer
dq2               out, a, scale, _gsize
// The third dequantization: scale and convert to half-
    precision floating-point
//// Pooling Instructions
pool.avg          out, x, w, _rq
// Integer 2D average pooling
pool.max          out, x
// Integer 2D max pooling
pool.min          out, x
// Integer 2D min pooling
pool.favg         out, x, w
// Floating-point 2D average pooling
pool.fmax         out, x
// Floating-point 2D max pooling
pool.fmin         out, x
// Floating-point 2D min pooling
roipool.avg       out, x, w, roi, _rq
// Integer ROI average pooling
roipool.favg      out, x, w, roi
// Floating-point ROI average pooling
roipool.max       out, x, roi
// Integer ROI max pooling
roipool.fmax      out, x, roi
// Floating-point ROI max pooling
roipool.min       out, x, roi
// Integer ROI min pooling
roipool.fmin      out, x, roi
// Floating-point ROI min pooling
dwconv            out, x, w, bias, _rq
// Integer depthwise convolution
fdwconv          out, x, w, bias
//Floating-point depthwise convolution
//// Convolution Instructions
conv              out, x, w, bias, _rq
// Integer convolution
conva             out, x, w, bias
// Integer convolution accumulate
fconv             out, x, w, bias
// Floating-point convolution
fconva            out, x, w, bias
// Floating-point convolution accumulate
//// CUBE Matrix Multiplication Instructions
mm.<nn|nt|tt>     out, x, w, bias, _rq, _relu
// Integer matrix multiplication
mma.<nn|nt|tt>    out, x, w, bias, _relu
// Integer matrix multiplication accumulate
fmm.<nn|nt|tt>    out, x, w, bias, _relu
// Floating-point matrix multiplication
fmma.<nn|nt|tt>   out, x, w, bias, _relu
//Floating-point matrix multiplication accumulate
```

Figure 28: Compute Instructions II.

```
//// Fully Connected Matrix Multiplication Instructions
fcmm.<nn|tn>    out, x, w, bias, _rq, _relu
// Integer matrix multiplication for fully connected
    layers
fcmma.<nn|tn>   out, x, w, bias, _relu
// Accumulated integer matrix multiplication for fully
    connected layers
ffcmm.<nn|tn>   out, x, w, bias, _relu
// Floating-point matrix multiplication for fully
    connected layers
ffcmma.<nn|tn> out, x, w, bias, _relu
// Accumulated floating-point matrix multiplication for
    fully connected layers
////Cross Comparison Instructions
fvcmax          out, a, b
// Element-wise cross comparison of two floating-point
    vectors to select the maximum value
fvcmin          out, a, b
// Element-wise cross comparison of two floating-point
    vectors to select the minimum value
vcmax           out, a, b
// Element-wise cross comparison of two integer vectors
    to select the maximum value
vcmin           out, a, b
// Element-wise cross comparison of two integer vectors
    to select the minimum value
////Data Copy and Reordering Instructions
cp              out, a
// Tensor copy instruction.
bc              out, a
// Broadcast: replicate a(n, 1, h, w) along the C
    dimension to form out(n, c, h, w)
cwtrans         out, a
// CW-dimension transpose instruction. Transposes the C
    and W dimensions of the input floating-point Tensor
    A and outputs the result to Tensor out.
wctrans         out, a
// WC-dimension transpose instruction. Transpose the W
    and C dimensions of the input floating-point Tensor
    A and outputs the result to Tensor out.
gather.pc       out, a, idx, cs, _bdlimit
// Per-channel gather instruction. Gather data from the W
     dimension of Tensor A into Tensor out based on the
    channel-shared index.
scatter.pc      out, a, idx
// Per-channel scatter instruction. Scatter data from the
     W dimension of Tensor A into Tensor out based on
    the channel-shared index.
gather2d.pc     out, a, idx, cs
// 2D gather instruction. Gather data from the H and W
    dimensions of Tensor A into Tensor out based on 2D
    coordinates shared per channel.
scatter2d.pc    out, a, idx
// 2D scatter instruction. Scatter data from the H and W
    dimensions of Tensor A into Tensor out based on 2D
    coordinates shared per channel.
gather          out, a, idx, cs
// Gather instruction. Gather data from the W dimension
    of Tensor A into Tensor out based on the provided
    index.
scatter         out, a, idx
// Scatter instruction. Scatter data from the W dimension
     of Tensor A into Tensor out based on the provided
    index.
hgatter         out, a, idx, cs
// H-dimension gather instruction. Gather data from the H
     dimension of Tensor A into Tensor out based on the
    provided index.
hscatter       out, a, idx
// H-dimension scatter instruction. Scatter data from the
     H dimension of Tensor A into Tensor out based on
    the provided index.
masksel         out_cnt, out, a, mask
// Mask selection instruction. Write elements from the W
    dimension of Tensor A to Tensor out at positions
    where the mask has a value of 1.
nzidx           out_cnt, out_idx, a
// Non-zero index generation instruction. Write the
    indices of non-zero elements from the W dimension of
     Tensor A into Tensor out.
```

Figure 29: Compute Instructions III.

```
sync.i          rs, _engine
// rs holds the tag. After executing this instruction,
     the tag is written into CSR.sync_tag.The CPU can
     read this register to determine whether the
     instruction has finished executing, and once read,
     the register is automatically cleared.
msgsend         rs
// Send the information in rs to the message queue.
msgwait         rs
// Block instruction issue until a message is received
     from the message queue, then store the message in rs
     .
```

Figure 30: Synchronization Instructions.